
Reducing Per-Sample Interference in Stochastic Optimization

Apostolos Avranas¹

Abstract

Modern optimizers combine gradients from the current mini-batch with historical optimization state, such as momentum or adaptive moments. While effective, this standard practice can produce parameter updates that actively increase the loss of individual samples. We term this phenomenon **per-sample interference** and propose redefining the parameter update as an optimization problem that explicitly minimizes it. Because the exact formulation of the problem is computationally prohibitive, we introduce a highly efficient surrogate. By reducing the problem’s dimensionality to the batch size and restricting the optimization to the last linear layer, we overcome memory and speed bottlenecks. This strategy hinges on our unexpected finding that this layer alone can reliably capture core second-order statistics of the full network. The resulting surrogate problem integrates readily into standard optimizers like SGD and AdamW, and can be solved using a small number of GPU-friendly iterations. Crucially, the method exhibits **favorable scaling properties**, as the relative computational overhead shrinks as the model size or input grows. Experiments on image classification benchmarks confirm *reduced per-sample interference* and *improved generalization*.

1. Introduction

Stochastic optimization is the backbone of modern deep learning. In practice, parameter updates are computed by aggregating gradients over a mini-batch and combining them with historical information such as momentum or adaptive moments. These mechanisms are crucial for stabilizing training, yet they implicitly assume that combining batch-averaged gradients with past optimization state is always beneficial for learning from the current mini-batch.

¹Amadeus, 821 Av. Jack Kilby, 06270, France. Correspondence to: Apostolos Avranas <apostolos.avranas@amadeus.com>.

In this work, we challenge this assumption. The interaction between historical optimization state and batch-averaged gradients can produce parameter updates that actively increase the loss of individual samples in the current batch. While this effect may be obscured at the batch level, it represents a systematic loss of information at the per-sample level. We refer to this phenomenon as *interference*. From this perspective, standard optimizers may inadvertently trade individual sample improvement for optimization stability.

To correct this imbalance, our goal is to protect each sample from the potentially conflicting influence of batch averaging and historical optimization states. We introduce a *novel* framework that **formalizes the parameter update as an optimization problem, explicitly minimizing the interfering influence on samples within the current mini-batch**.

Unfortunately, directly solving the resulting optimization problem is practically prohibitive. We employ a series of steps to derive a surrogate problem admitting a computationally efficient solution:

- *Dimensionality reduction*: We mathematically transform the optimization problem out of parameter space and into the drastically smaller batch space.
- *Last-layer approximation*: The formulation involves the full-network per-sample gradients whose computation adds a massive overhead. We demonstrate that with careful manipulation, the problem can be approximated using the per-sample gradients of only the last linear layer. This relies on our surprising finding that this layer alone reliably captures the core second-order statistics of the full-network per-sample gradients, making the surrogate problem highly representative of the original.
- *Implicit gradient computation*: Even for the last layer, materializing per-sample gradients poses a bottleneck. We leverage Khatri-Rao factorization properties to bypass materializing these gradients, completely eliminating the associated memory and speed overheads.
- *Efficient solver*: We propose a tailored solver for the resulting surrogate problem that rapidly converges using a small number of GPU-friendly iterations.

Overall, our framework is optimizer-agnostic and can be incorporated into widely used methods such as Stochastic Gradient Descent (SGD) with momentum and AdamW. Im-

portantly, the introduced computational and memory overhead is small and naturally shrinks as the model size or input dimensions scale up. Empirically, we confirm that the proposed method successfully mitigates interference to individual samples in the current batch. Furthermore, evaluations on image classification benchmarks show improved generalization, suggesting that controlling update interference is a practical and complementary dimension for improving deep learning optimization.

2. Literature Review

Our work is most closely related to research in continual learning, where the objective is to prevent catastrophic forgetting of previously acquired knowledge when learning new tasks. In contrast, our setting is fundamentally different: rather than protecting past knowledge from new data, we aim to prevent the interfering influence of past optimization state on the incoming data.

Within the continual learning literature, the closest method to our approach is Gradient Episodic Memory (GEM) (Lopez-Paz & Ranzato, 2017), which uses a similar notion of interference that is based on gradient alignment. To determine the update direction, GEM formulates an optimization problem whose dimensionality is reduced from the parameter space to the number of samples. However, challenges such as reliance on large episodic memories and the efficient solution of the resulting optimization problem are not fully addressed. Subsequent works (Chaudhry et al., 2019; Hu et al., 2020) alleviate some of these limitations by replacing episodic memories with averaged gradients over past samples. Related approaches enforce orthogonality or projection-based constraints between gradients of different tasks (Farajtabar et al., 2020; Saha et al., 2021). Finally, recent work (Luo et al., 2025) shows that catastrophic forgetting effects persist in contemporary large language models, with severity increasing as model size grows.

By viewing each sample within the current mini-batch as a distinct task, we can draw parallels with multi-task learning (MTL). In (Navon et al., 2022), each task is modeled as a player in a game, leading to an optimization problem that learns weights over per-task gradients and relies on a task-task Gram matrix capturing pairwise gradient interactions. In (Yu et al., 2020), the authors use a notion of inter-task interference similar to ours and, analogously to (Chaudhry et al., 2019), propose projecting averaged gradients to reduce conflicts across tasks. In (Dong et al., 2022), orthogonal decomposition is used to avoid interference between gradients of different tasks. In (Liu et al., 2021a), they consider weighting the gradients of each task so as to minimize the worst conflict. In (Liu et al., 2021b), the weights of each task aim to equalize the learning speed across tasks while in (Senushkin et al., 2023), they aim to

construct orthonormal gradients between different tasks.

In contrast to the above works, (Yang et al., 2023) explicitly accounts for the base optimizer and proposes adaptations for the MTL setting. Similarly, our goal is to adapt practical optimizers to alter the optimization trajectory in a way that reduces, at each iteration, the harmful impact on the samples of the current mini-batch. In this work we focus on two widely used optimizers: AdamW (Loshchilov & Hutter, 2019) and SGD with momentum (Sutskever et al., 2013; Nesterov, 1983). AdamW is a variant of Adam (Kingma & Ba, 2014) that decouples weight decay from the gradient-based update. Other strong Adam-style methods include Adafactor (Shazeer & Stern, 2018; Zhai et al., 2022), which reduces memory usage, LAMB (You et al., 2020), which employs layer-wise adaptation, and RAdam (Liu et al., 2020), which corrects the variance of the adaptive learning rate. For SGD-style methods, competitive optimizers used in large-scale training include MUON (Jordan et al., 2024), which encourages orthogonal updates, and Lion (Chen et al., 2023), which uses the sign of momentum.

An important element of this work is working with per-sample gradients without incurring excessive memory and computational costs. Per-sample gradients and statistics like their norms are useful in various contexts like differential privacy (Abadi et al., 2016), predicting generalization (Yin et al., 2018) and developing new optimizers (Roulet & Agarwala, 2025). Unfortunately, computing per-sample gradients is demanding. For convolutional neural networks, (Rochette et al., 2019) propose a method to compute them more efficiently. Also libraries like Opacus (Yousefpour et al., 2021) and Backpack (Dangel et al., 2020) were developed for that purpose. We note that computing full per-sample gradients is costly in both memory and runtime. However, an efficient technique exists to compute their norms without explicitly calculating the full gradients. This trick was used in (Roulet & Agarwala, 2025; Li et al., 2022; Bu et al., 2023).

3. Motivation and Problem Formulation

Let $F(\mathbf{x}; \boldsymbol{\theta})$ denote a model with parameters $\boldsymbol{\theta} \in \mathbb{R}^\Theta$ with input $\mathbf{x}$. Standard training proceeds in mini-batches of B samples $\{\mathbf{x}_i\}_{i=1}^B$, each with a per-sample loss $\mathcal{L}_i(\boldsymbol{\theta}) = \mathcal{L}(F(\mathbf{x}_i; \boldsymbol{\theta}))$. In practice, standard optimizers such as SGD and AdamW (Sutskever et al., 2013; Kingma & Ba, 2014; Loshchilov & Hutter, 2019) update parameters using only the *aggregated* batch gradient $\mathbf{g} = \nabla_{\boldsymbol{\theta}} \frac{1}{B} \sum_{i=1}^B \mathcal{L}_i(\boldsymbol{\theta})$, combined with an internal optimization state (e.g., momentum). While this process reliably decreases the *average* loss, the ultimate goal is typically to perform well on *every* sample. This discrepancy is particularly evident in classification tasks, where cross-entropy is a surrogate for accuracy. Minimizing the mean loss does not guarantee improvement for every sample and may even fail to increase the number of

correctly classified examples.

Beyond this mismatch in objectives, batch aggregation also discards valuable information about the batch: we lose visibility into per-sample contributions and conflicts, i.e., which examples are helped or harmed by a parameter update.

Finally, practical optimizers blend the current batch gradient with historical information, such as momentum or adaptive moments. Consequently, the actual parameter update may deviate significantly from the descent direction of the current batch, potentially increasing the loss for a subset of the *current* samples. In this sense, the update can be “catastrophically” harmful to the current batch samples. This motivates a per-sample view of the update: rather than considering only the aggregated batch gradient, we seek robust updates that respect the descent direction of each sample.

Let $\mathbf{G} \in \mathbb{R}^{B \times \Theta}$ be the matrix of per-sample gradients, where each row $\mathbf{G}_i = \frac{1}{B} \nabla_{\theta} \mathcal{L}_i(\theta)$ corresponds to the gradient of the i -th sample. Using a first-order Taylor expansion, the change in loss for sample i is:

$$\mathcal{L}_i(\theta + \Delta\theta) \approx \mathcal{L}_i(\theta) + B\mathbf{G}_i^T \Delta\theta \quad (1)$$

When $\mathbf{G}_i^T \Delta\theta > 0$, the loss for sample i increases. We term this increase as **interference**. According to the Taylor’s first order approximation, the severity of the interference is proportional to $\max(0, \mathbf{G}_i^T \Delta\theta)$. Ideally, we seek a direction $\Delta\theta$ that improves *every* sample simultaneously. In practice, however, conflicting gradients make this *impossible*.

Since we cannot guarantee that no sample is interfered with, our goal shifts to minimizing the total interference across the mini-batch. We construct an objective $J(\Delta\theta)$ that linearly penalizes sample interference (i.e., when $\mathbf{G}_i^T \Delta\theta > 0$) while assigning zero penalty to directions that successfully decrease a sample’s loss ($\mathbf{G}_i^T \Delta\theta \leq 0$). Defining the helper function:

$$H(\mathbf{v}) := \sum_{i=1}^B \max(0, [\mathbf{v}]_i),$$

where $[\mathbf{v}]_i$ denotes the i -th element of vector $\mathbf{v} \in \mathbb{R}^B$, we can express our objective as $J(\Delta\theta) := H(\mathbf{G}\Delta\theta)$. We therefore seek the optimal parameter update:

$$\Delta\theta^* = \arg \min_{\Delta\theta} J(\Delta\theta). \quad (2)$$

3.1. Introducing a Constraint on the Update Direction

Solving Equation (2) directly presents several pitfalls. First, the trivial solution $\Delta\theta = \mathbf{0}$ is technically optimal (zero interference) but ineffective for training the model. Second, Equation (2) relies solely on the gradients of the current batch. In contrast, modern practical optimizers compute at each iteration a parameter update $\Delta\theta_b$ that incorporates information from previous iterations (e.g., via momentum

for SGD or adaptive moments for Adam/AdamW). This historical context is crucial for stabilizing the optimization trajectory. Deviating significantly from the standard update $\Delta\theta_b$ discards this valuable information.

To address those issues, we constrain the proposed update $\Delta\theta$ to remain within the vicinity of the standard optimizer’s update $\Delta\theta_b$. Formally, we restrict $\Delta\theta$ to lie within a ball (measured by the Euclidean norm $\|\cdot\|$) centered at $\Delta\theta_b$, with a radius proportional to the magnitude of $\Delta\theta_b$:

$$\Delta\theta^* = \arg \min_{\Delta\theta} H(\mathbf{G}\Delta\theta) \text{ s.t. } \|\Delta\theta - \Delta\theta_b\| \leq c \|\Delta\theta_b\|, \quad (3)$$

where $c > 0$ is a hyperparameter that controls the radius of the *trust region*. This formulation ensures that while we actively seek to minimize the interference to individual samples, we do so without negating the general descent direction $\Delta\theta_b$ determined by the global loss and the optimizer’s state.

Importantly, the optimization problem in (3) is convex, ensuring it can be solved efficiently. While other formalizations could be employed to mitigate sample interference, alternative designs require careful consideration. For instance, modifying the objective to minimize the discrete count of interfered-with samples renders the problem combinatorial and non-convex. Furthermore, in our initial experiments, we explored a GEM-like formulation (Lopez-Paz & Ranzato, 2017), solving $\min_{\Delta\theta} \|\Delta\theta - \Delta\theta_b\|^2$ s.t. $\mathbf{G}_i^T \Delta\theta \leq 0 \forall i$. However, we found its constraint to be often overly restrictive, forcing the solution $\Delta\theta$ substantially far from the reference $\Delta\theta_b$ and causing severe optimization instability.

4. Challenges

Equation (3) provides a principled framework for reducing per-sample interference. However, directly integrating it into a standard training loop presents several practical challenges:

- **High Dimensionality:** The variable $\Delta\theta$ resides in $\mathbb{R}^{\Theta}$. Since Θ is typically in the millions or billions for modern models, performing constrained optimization over such a high-dimensional space is computationally prohibitive.
- **Per-Sample Gradient Cost:** Equation (3) involves the per-sample gradient matrix $\mathbf{G}$. Computing and storing it incurs massive memory and runtime overheads. It is crucial to avoid explicitly materializing this matrix.
- **Solver Efficiency:** The solver for Equation (3) must be highly efficient and GPU-friendly to ensure that the training throughput is not significantly compromised.

In the following subsections, we address these challenges systematically.

4.1. High Dimensionality of Optimization Variables

The dimensionality of the problem can be reduced from the number of parameters Θ to the batch size B (note $B \ll \Theta$).

Proposition 4.1. *Let $\mathcal{S}^*$ denote the set of optimal solutions of the optimization problem (3). Then:*

1. *There exists an optimal solution $\Delta\theta^* \in \mathcal{S}^*$ such that:*

$$\Delta\theta^* = \Delta\theta_b + \mathbf{G}^\top \mathbf{w}, \quad (4)$$

where $\mathbf{w} \in \mathbb{R}^B$ and $\mathbf{w} \leq \mathbf{0}$.

2. *Among all optimal solutions $\Delta\theta \in \mathcal{S}^*$, the unique solution that minimizes the distance to the update $\Delta\theta_b$ (i.e., $\min_{\Delta\theta \in \mathcal{S}^*} \|\Delta\theta - \Delta\theta_b\|$) satisfies the form given in Equation (4).*

Proof. See Appendix A for the proof. $\square$

Proposition 4.1 allows us to optimize over $\mathbf{w} \in \mathbb{R}^B$ instead of $\Delta\theta \in \mathbb{R}^\Theta$. Substituting $\Delta\theta = \Delta\theta_b + \mathbf{G}^\top \mathbf{w}$ into Equation (3), the optimization problem transforms into:

$$\begin{aligned} \mathbf{w}^* = \arg \min_{\mathbf{w} \in \mathbb{R}^B} H(\mathbf{Q}\mathbf{w} + \mathbf{G}\Delta\theta_b) \\ \text{s.t. } \mathbf{w}^\top \mathbf{Q}\mathbf{w} \leq c^2 \|\Delta\theta_b\|^2, \mathbf{w} \leq \mathbf{0}, \end{aligned} \quad (5)$$

where $\mathbf{Q} = \mathbf{G}\mathbf{G}^\top \in \mathbb{R}^{B \times B}$ is the Gram matrix (symmetric and positive semi-definite) of the per-sample gradients.

A crucial property of the problem's defining quantities (i.e. $\mathbf{Q}$, $\mathbf{G}\Delta\theta_b$, and $\|\Delta\theta_b\|^2$) is their **layer-wise decomposability**. Consider a network composed of L layers, $F = F_L \circ F_{L-1} \circ \dots \circ F_1$, where each layer $F_l(\cdot; \theta_l)$ has its distinct parameters $\theta_l \in \mathbb{R}^{\Theta_l}$. Let $\mathbf{G}_l \in \mathbb{R}^{B \times \Theta_l}$ denote the per-sample gradient matrix for layer l , and $\Delta\theta_{b,l} \in \mathbb{R}^{\Theta_l}$ the corresponding slice of the update vector $\Delta\theta_b$. With the full gradient matrix $\mathbf{G} = [\mathbf{G}_1, \dots, \mathbf{G}_L]$ and partial Gram matrix $\mathbf{Q}_l = \mathbf{G}_l \mathbf{G}_l^\top$, the defining quantities decompose as:

$$\mathbf{Q} = \sum_{l=1}^L \mathbf{Q}_l, \quad \mathbf{G}\Delta\theta_b = \sum_{l=1}^L \mathbf{G}_l \Delta\theta_{b,l}, \quad \|\Delta\theta_b\|^2 = \sum_{l=1}^L \|\Delta\theta_{b,l}\|^2.$$

4.2. Naive solutions do not work

Although Equation (5) mitigates the dimensionality bottleneck, the strict dependence on per-sample gradients remains, required *both* to form the reduced problem (due to $\mathbf{Q}$ and $\mathbf{G}\Delta\theta_b$ involving $\mathbf{G}$) and to reconstruct the update $\Delta\theta^* = \Delta\theta_b + \mathbf{G}^\top \mathbf{w}^*$.

Before deriving our solution, we note that two straightforward implementation strategies fail in practice:

- **Memory-prohibitive approach (store full $\mathbf{G}$).** Compute and store $\mathbf{G}$. Solve Equation (5) for $\mathbf{w}^*$, and finally compute

$\mathbf{G}^\top \mathbf{w}^*$. In practice, this approach is only feasible for very small models, as it scales memory by a factor of B .

- **Slow approach (second backward).** During the backward pass, accumulate the quantities $\mathbf{Q}_l$, $\mathbf{G}_l \Delta\theta_{b,l}$, and $\|\Delta\theta_{b,l}\|^2$ per layer (without storing $\mathbf{G}_l$) to form Equation (5). Solve for $\mathbf{w}^*$ and obtain $\mathbf{G}^\top \mathbf{w}^*$ via a second backward pass on the weighted loss $\frac{1}{B} \sum_{i=1}^B [\mathbf{w}^*]_i \mathcal{L}_i$. Beyond requiring custom backward kernels at each layer to produce these quantities, the second backward pass adds significant runtime overhead.

In the next subsection, we introduce an approximation that avoids explicitly forming $\mathbf{G}$ while still enabling efficient construction of Equation (5).

4.3. Approximating the Full Quantities via Last Layers

We avoid requiring the full per-sample gradient matrix $\mathbf{G}$ by restricting computations to the last few layers (from ℓ to L) and utilizing the partial quantities $\mathbf{Q}_{\ell:L} = \sum_{k=\ell}^L \mathbf{G}_k \mathbf{G}_k^\top$, $(\mathbf{G}\Delta\theta_b)_{\ell:L} = \sum_{k=\ell}^L \mathbf{G}_k \Delta\theta_{b,k}$, and $\|\Delta\theta_{b,\ell:L}\|^2 = \sum_{k=\ell}^L \|\Delta\theta_{b,k}\|^2$. Specifically we propose the simple approximation that the Gram matrix of the full network is proportional to that of the last layers:

$$\mathbf{Q} \approx \alpha \mathbf{Q}_{\ell:L}, \quad (6)$$

for an unknown scaling factor $\alpha > 0$.

Remarkably, under vanilla SGD (i.e. $\Delta\theta_b = -\text{lr} \cdot \mathbf{G}^\top \mathbf{1}$ where $\mathbf{1}$ is the all-ones vector), this single assumption propagates to the remaining quantities:

$$\mathbf{G}\Delta\theta_b = -\text{lr} \mathbf{Q} \mathbf{1} \approx \alpha (-\text{lr} \mathbf{Q}_{\ell:L} \mathbf{1}) = \alpha (\mathbf{G}\Delta\theta_b)_{\ell:L},$$

$$\|\Delta\theta_b\|^2 = \text{lr}^2 \mathbf{1}^\top \mathbf{Q} \mathbf{1} \approx \alpha (\text{lr}^2 \mathbf{1}^\top \mathbf{Q}_{\ell:L} \mathbf{1}) = \alpha \|\Delta\theta_{b,\ell:L}\|^2.$$

What makes this result powerful is that when we inject these scaled approximations into Equation (5), **the unknown factor α completely vanishes** from both the objective function and the constraints. The approximated surrogate problem is invariant to α , mapping perfectly back to Equation (5) but with all defining quantities replaced by their partial, last-layer counterparts:

$$\begin{aligned} \mathbf{w}^* = \arg \min_{\mathbf{w} \in \mathbb{R}^B} H(\mathbf{Q}_{\ell:L} \mathbf{w} + (\mathbf{G}\Delta\theta_b)_{\ell:L}) \\ \text{s.t. } \mathbf{w}^\top \mathbf{Q}_{\ell:L} \mathbf{w} \leq c^2 \|\Delta\theta_{b,\ell:L}\|^2, \mathbf{w} \leq \mathbf{0}, \end{aligned} \quad (7)$$

To confirm the surprising robustness of this approximation we conduct the following experiments:

Experiment: Approximation Quality of $\mathbf{Q}$. We measure how much information about the full-network *second-order statistics* $\mathbf{Q}$ is retained by the last layers, i.e., how accurate Equation (7) is. To evaluate this, we compute the full per-sample gradients $\mathbf{G}$ and with that $\mathbf{Q}$ and $\mathbf{Q}_{\ell:L}$ for every

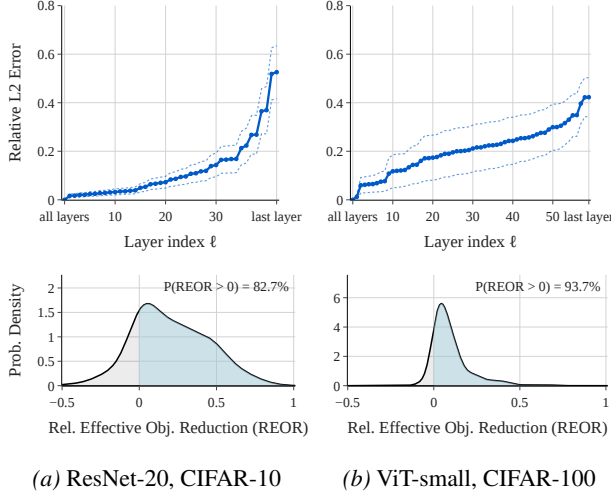


Figure 1. Approximation quality at epoch 100. **Top Row:** Relative L_2 error of the Gram matrix approximation, $\frac{\|\mathbf{Q} - \alpha \mathbf{Q}_{\ell:L}\|_F}{\|\mathbf{Q}\|_F}$, as a function of included layers (lower is better). The solid curve shows the mean, and the two dotted curves show one standard deviation above and below the mean at each point. **Bottom Row:** Distribution of the relative *effective objective reduction* (REOR), $\frac{J(\Delta\theta_b) - J(\Delta\theta^*)}{J(\Delta\theta_b)}$, on the original problem (higher is better).

layer ℓ . In the top row of Figure 1 we depict the relative Frobenius error $\|\mathbf{Q} - \alpha \mathbf{Q}_{\ell:L}\|_F / \|\mathbf{Q}\|_F$, where α is chosen to minimize this error; this scaling is appropriate because the surrogate problem is invariant to α (details in Section F). The figures reveal that the error diminishes rapidly as earlier layers are included. Even restricting solely to the final linear layer ($\ell = L$) retains significant structural information, maintaining an average error of around 0.5 or less.¹

Experiment: End-to-End Interference Reduction. Our ultimate goal is to incorporate per-sample interference reduction into advanced optimizers (SGD with momentum, AdamW). However, their intricate update rules make applying the last-layer approximation in a strict theoretical manner intractable. To circumvent this, we propose using Equation (7) as a general surrogate and adapt it for other optimizers. Although Equation (7) is derived assuming vanilla SGD, and applying it elsewhere introduces an inconsistency, we show in practice that the resulting adaptations (Equation (10) for SGD with momentum and Equation (14) for AdamW, detailed in Section 4.4), remain faithful proxies for our ultimate objective of reducing total per-sample interference (Equation (2)), even when restricting the approximation to the final layer ($\ell = L$). We quantify the entire pipeline’s effectiveness via the *relative effective objective reduction* (REOR): $(J(\Delta\theta_b) - J(\Delta\theta^*)) / J(\Delta\theta_b)$, where $\Delta\theta^* = \Delta\theta_b + \mathbf{G}^\top \mathbf{w}^*$ and $\mathbf{w}^*$ is the solution to the respective

¹We note that for the ViT model, the matrices incorporate AdamW weighting, i.e. $\mathbf{Q}_1$ and $\mathbf{Q}_{1,\ell:L}$ (defined in Section 4.4).

problem. Despite the underlying simplifications, the bottom row of Figure 1 shows that the majority of updates result in positive reductions, confirming that the optimized $\Delta\theta^*$ is systematically less interfering than the standard update $\Delta\theta_b$. Additional figures and details are provided in Section F.

4.4. Adaptation to Modern Optimizers

In this section, we extend Equation (7) to modern optimizers: SGD with momentum and AdamW. A critical requirement for these adaptations is to completely avoid the explicit materialization of $\mathbf{G}$ at any stage, including when computing the final update $\Delta\theta^* = \Delta\theta_b + \mathbf{G}^\top \mathbf{w}^*$. We structure our surrogate problems so that their optimal solution $\mathbf{w}^*$ allows $\Delta\theta^*$ to be implicitly realized. Specifically, we aim to compute the update via a standard backward pass on a weighted loss, $\sum_{i=1}^B [\mathbf{w}^*]_i \mathcal{L}_i$, naturally leveraging the underlying optimizer. Finally, as we advocate restricting the approximation to the very last layer, we henceforth drop the $\ell:L$ subscript and simply denote it by L .

SGD with Momentum: We consider the standard SGD formulation used in deep learning (Sutskever et al., 2013):

$$\begin{aligned} \mathbf{g}_t &\leftarrow \mathbf{g}_t + \text{wd} \cdot \boldsymbol{\theta}_t, \\ \mathbf{v}_t &\leftarrow \text{mom} \cdot \mathbf{v}_{t-1} + \mathbf{g}_t, \\ \boldsymbol{\theta}_{t+1} &\leftarrow \boldsymbol{\theta}_t - \text{lr} \cdot \mathbf{v}_t, \end{aligned}$$

where wd is weight decay, mom is momentum factor, lr is the learning rate, and $\mathbf{v}_t$ is the momentum vector. Hence, the standard update $\Delta\theta_b = \boldsymbol{\theta}_{t+1} - \boldsymbol{\theta}_t$ is equal to:

$$\Delta\theta_b = -\text{lr} \cdot \mathbf{v}_t = -\text{lr} \cdot (\text{mom} \cdot \mathbf{v}_{t-1} + \text{wd} \cdot \boldsymbol{\theta}_t + \mathbf{g}_t). \quad (8)$$

Since $\mathbf{g}_t = \mathbf{G}^\top \mathbf{1}$, the target term $\mathbf{G}^\top \mathbf{w}^*$ can be **automatically** incorporated by adjusting the loss to:

$$\mathcal{L}_{\mathbf{w}^*} = \sum_{i=1}^B \left(1 - \frac{[\mathbf{w}^*]_i}{\text{lr}}\right) \mathcal{L}_i. \quad (9)$$

This is because the aggregated gradients of $\mathcal{L}_{\mathbf{w}^*}$ are $\tilde{\mathbf{g}}_t = \mathbf{g}_t - \frac{1}{\text{lr}} \mathbf{G}^\top \mathbf{w}^*$ and substituting $\mathbf{g}_t$ with $\tilde{\mathbf{g}}_t$ in Equation (8) gives the desired updates $\Delta\theta^* = \Delta\theta_b + \mathbf{G}^\top \mathbf{w}^*$.

Finally, to make the formulation clearer, we perform the change of variables $\mathbf{w} \leftarrow -\frac{1}{\text{lr}} \mathbf{w}$ so that the resulting optimization problem directly yields the *additional relative weight* assigned to each sample. Substituting this into Equation (5), we obtain the optimization problem for SGD:

$$\begin{aligned} \arg \min_{\mathbf{w} \in \mathbb{R}^B} \quad & H\left(-\mathbf{Q}_L \mathbf{w} + \frac{1}{\text{lr}} \mathbf{G}_L \Delta\theta_{b,L}\right) \\ \text{s.t.} \quad & \mathbf{w}^\top \mathbf{Q}_L \mathbf{w} \leq \left(\frac{c}{\text{lr}}\right)^2 \|\Delta\theta_{b,L}\|^2, \quad \mathbf{0} \leq \mathbf{w} \leq \mathbf{2} \end{aligned} \quad (10)$$

Because the problem relies on the last-layer approximation, we add the constraint $\mathbf{w} \leq \mathbf{2}$ (where $\mathbf{2} = 2\mathbf{1}$) as a safeguard

to ensure that no sample is unreasonably weighted. After solving Equation (10), the interference-reducing parameter update $\Delta\theta^*$ can be implicitly computed using the standard SGD optimizer on the weighted loss:

$$\mathcal{L}_{w^*} = \sum_{i=1}^B (1 + [w^*]_i) \mathcal{L}_i \quad (11)$$

AdamW: The AdamW (Loshchilov & Hutter, 2019) optimizer updates are:

$$\begin{aligned} \mathbf{m}_t &\leftarrow \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t, \\ \mathbf{v}_t &\leftarrow \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2, \\ \theta_{t+1} &\leftarrow \theta_t - \text{lr} \cdot \left(\frac{\mathbf{m}_t / (1 - \beta_1^t)}{\sqrt{\mathbf{v}_t / (1 - \beta_2^t)} + \varepsilon} + \text{wd} \cdot \theta_t \right), \end{aligned}$$

where ε is a small constant, and all divisions, squaring, and square roots are performed element-wise. The standard update $\Delta\theta_b = \theta_{t+1} - \theta_t$ is given by:

$$\Delta\theta_b = -\text{lr} \cdot \left(\text{wd} \cdot \theta_t + \frac{\beta_1}{(1 - \beta_1)} \kappa \odot \mathbf{m}_{t-1} + \kappa \odot \mathbf{g}_t \right) \quad (12)$$

with $\kappa = \frac{(1 - \beta_1)/(1 - \beta_1^t)}{\sqrt{\mathbf{v}_t/(1 - \beta_2^t)} + \varepsilon}$ and $\odot$ the Hadamard product.

Optimizing here the weighted loss of Equation (11), creates gradients $\tilde{\mathbf{g}}_t = \mathbf{g}_t + \mathbf{G}^\top \mathbf{w}^*$ which produce the modified second moment $\tilde{\mathbf{v}}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \tilde{\mathbf{g}}_t^2$. Since β_2 is typically very close to 1 (e.g., 0.999), we can safely assume $\tilde{\mathbf{v}}_t \approx \mathbf{v}_t$, and consequently the modified scaling factor $\tilde{\kappa} \approx \kappa$. Therefore, the weighted loss results in the updates:

$$\begin{aligned} \Delta\theta &= -\text{lr} \cdot \text{wd} \cdot \theta_t - \text{lr} \cdot \frac{\beta_1}{(1 - \beta_1)} \tilde{\kappa} \odot \mathbf{m}_{t-1} - \text{lr} \cdot \tilde{\kappa} \odot \tilde{\mathbf{g}}_t \\ &\approx \Delta\theta_b - \text{lr} \cdot \mathbf{D}_\kappa \mathbf{G}^\top \mathbf{w}^*, \end{aligned} \quad (13)$$

where $\mathbf{D}_\kappa = \text{diag}(\kappa)$. Unlike SGD, the element-wise division by $\sqrt{\tilde{\mathbf{v}}_t}$ introduces this diagonal scaling $\mathbf{D}_\kappa$, meaning the optimal form described in Proposition 4.1 no longer strictly applies. To proceed, we revisit our initial optimization problem (3) and directly substitute $\Delta\theta$ using Equation (13). Given the structural similarities with SGD case, we choose to retain the non-negativity constraint $\mathbf{w} \geq \mathbf{0}$, alongside the bounding constraint $\mathbf{w} \leq \mathbf{2}$ ensuring reasonable weighting. Incorporating also the last-layer restriction the AdamW surrogate problem is:

$$\begin{aligned} &\arg \min_{\mathbf{w} \in \mathbb{R}^B} H \left(-\mathbf{Q}_{1,L} \mathbf{w} + \frac{1}{\text{lr}} \mathbf{G}_L \Delta\theta_{b,L} \right) \\ &\text{s.t. } \mathbf{w}^\top \mathbf{Q}_{2,L} \mathbf{w} \leq \left(\frac{c}{\text{lr}} \right)^2 \|\Delta\theta_{b,L}\|^2, \mathbf{0} \leq \mathbf{w} \leq \mathbf{2} \end{aligned} \quad (14)$$

where $\mathbf{Q}_{1,L} = \mathbf{G}_L \mathbf{D}_{\kappa,L} \mathbf{G}_L^\top$ and $\mathbf{Q}_{2,L} = \mathbf{G}_L \mathbf{D}_{\kappa,L}^2 \mathbf{G}_L^\top$ are the weighted partial Gram matrices, $\mathbf{D}_{\kappa,L} = \text{diag}(\kappa_L)$ and κ_L refers to the partial of κ corresponding to the L -th layer.

4.5. The Final Nail for Per-Sample Gradients

For linear layers, we can compute the defining quantities of our optimization problems (Equations (10) and (14)) without explicitly materializing the per-sample gradients. Combined with our last-layer approximation strategy ($\ell = L$), which in almost all state-of-the-art architectures is a linear projection, this property means we can **completely bypass the expensive computation and storage of $\mathbf{G}$, or any of its partials**.

Assume the L -th layer is a linear layer with parameters $\theta_L \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, receiving input $\mathbf{x}_L \in \mathbb{R}^{B \times d_{\text{in}}}$ and outputs $\mathbf{y}_L = \mathbf{x}_L \theta_L^\top$. Let $\mathbf{g}_{\mathbf{y}_L} = \nabla_{\mathbf{y}_L} \mathcal{L} \in \mathbb{R}^{B \times d_{\text{out}}}$ denote the gradient of the loss with respect to these outputs. The per-sample gradient is $\mathbf{G}_L = \mathbf{g}_{\mathbf{y}_L} \otimes \mathbf{x} \in \mathbb{R}^{B \times (d_{\text{out}} d_{\text{in}})}$, where $\otimes$ here denotes the Khatri-Rao (i.e., row-wise Kronecker) product. Using the properties of this product, the partial Gram matrix $\mathbf{Q}_L = \mathbf{G}_L \mathbf{G}_L^\top$ can be factorized as:

$$\mathbf{Q}_L = (\mathbf{x}_L \mathbf{x}_L^\top) \odot (\mathbf{g}_{\mathbf{y}_L} \mathbf{g}_{\mathbf{y}_L}^\top).$$

Computing $\mathbf{Q}_L$ through this factorization elegantly eliminates the need to materialize $\mathbf{G}_L$, circumventing the prohibitive $\mathcal{O}(B \cdot d_{\text{in}} \cdot d_{\text{out}})$ memory cost. Furthermore, it reduces the computational time complexity of forming $\mathbf{Q}_L$ from $\mathcal{O}(B^2 \cdot d_{\text{in}} \cdot d_{\text{out}})$ to just $\mathcal{O}(B^2(d_{\text{in}} + d_{\text{out}}))$.

For AdamW, the inclusion of the diagonal scaling $\mathbf{D}_{\kappa,L}$ makes the exact factorization of $\mathbf{Q}_{1,L} = \mathbf{G}_L \mathbf{D}_{\kappa,L} \mathbf{G}_L^\top$ impossible in a single step². To resolve this, we first factorize $\kappa_L \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ using a low-rank Singular Value Decomposition (SVD). Let $\kappa_L \approx \mathbf{U}^r \mathbf{S}^r \mathbf{V}^{r\top}$, where $\mathbf{U}^r \in \mathbb{R}^{d_{\text{out}} \times r}$, $\mathbf{V}^r \in \mathbb{R}^{d_{\text{in}} \times r}$, and $\mathbf{S}^r = \text{diag}(\sigma_1, \dots, \sigma_r)$ contains the r largest singular values ($r \leq \min(d_{\text{in}}, d_{\text{out}})$). Denoting the k -th columns of $\mathbf{U}^r$ and $\mathbf{V}^r$ by $\mathbf{u}_k$ and $\mathbf{v}_k$ respectively, we approximate $\mathbf{Q}_{1,L}$ as:

$$\mathbf{Q}_{1,L}^r = \sum_{k=1}^r \sigma_k (\mathbf{x}_L \text{diag}(\mathbf{v}_k) \mathbf{x}_L^\top) \odot (\mathbf{g}_{\mathbf{y}_L} \text{diag}(\mathbf{u}_k) \mathbf{g}_{\mathbf{y}_L}^\top).$$

This approximation becomes exact (i.e. $\mathbf{Q}_{1,L} = \mathbf{Q}_{1,L}^r$) when r equals the rank of κ_L . In practice, however, using even $r \leq 5$ is sufficient to bound the relative Frobenius error $\|\mathbf{Q}_{1,L} - \mathbf{Q}_{1,L}^r\|_F / \|\mathbf{Q}_{1,L}\|_F$ to less than 2%.

In Section B, we demonstrate that as batch sizes or layer dimensions scale, this Khatri-Rao methodology yields *order-of-magnitude savings in both memory and runtime for both optimizers*. The appendix also provides further intuition for the SVD approximation, the corresponding PyTorch code, detailed profiling metrics, and an explanation of how the remaining quantity $\mathbf{G}_L \Delta\theta_{b,L}$ is efficiently computed without materializing $\mathbf{G}_L$.

²The same approximation applies directly to $\mathbf{Q}_{2,L}$ by replacing κ_L with κ_L^2 .

4.6. Efficient Solver for the Optimization Problem

The optimization problem we need to solve has the form:

$$\mathbf{w}^* = \arg \min_{\mathbf{w} \in \mathcal{C}} H(-\mathbf{Q}_1 \mathbf{w} + \mathbf{q}) \quad (15)$$

where $\mathcal{C} = \{\mathbf{w} \in [0, 2]^B \mid \mathbf{w}^\top \mathbf{Q}_2 \mathbf{w} \leq r\}$ is the domain, $\mathbf{Q}_1, \mathbf{Q}_2 \in \mathbb{R}^{B \times B}$ are symmetric positive semi-definite matrices (with $\mathbf{Q}_1 = \mathbf{Q}_2$ for SGD case), $\mathbf{q} \in \mathbb{R}^B$, and $r > 0$.

Despite being convex, solving Equation (15) efficiently is non-trivial due to: (i) the non-smoothness of H ; (ii) the complex feasible domain arising from the intersection of an ellipsoidal constraint with bounding box constraints; and (iii) the necessity of a fully GPU-resident routine.

To address these challenges, we choose the Chambolle–Pock (CP) primal–dual algorithm (Chambolle & Pock, 2011), as it is particularly well-suited for non-smooth optimization under simple convex constraints.³ In brief, the CP algorithm selects step sizes $\tau, \sigma > 0$ satisfying $\tau\sigma\|\mathbf{Q}_1\|_\sigma^2 < 1$, where $\|\mathbf{Q}_1\|_\sigma$ denotes the spectral norm of $\mathbf{Q}_1$, and an extrapolation parameter $\theta \in [0, 1]$. Each iteration takes the form:

$$\begin{aligned} \mathbf{y}^{k+1} &= \Pi_{[0,1]^B}(\mathbf{y}^k + \sigma(-\mathbf{Q}_1 \tilde{\mathbf{w}}^k + \mathbf{q})) \\ \mathbf{w}^{k+1} &= \Pi_{\mathcal{C}}(\mathbf{w}^k + \tau \mathbf{Q}_1 \mathbf{y}^{k+1}) \\ \tilde{\mathbf{w}}^{k+1} &= \mathbf{w}^{k+1} + \theta(\mathbf{w}^{k+1} - \mathbf{w}^k) \end{aligned}$$

Here, $\Pi_{[0,1]^B}$ denotes the Euclidean projection onto the unit hypercube $[0, 1]^B$, which can be trivially computed by an element-wise clipping operation (time complexity $\mathcal{O}(B)$). In contrast, computing the projection $\Pi_{\mathcal{C}}$ onto the domain $\mathcal{C}$ is more involved; nevertheless, it can be adequately approximated in $\mathcal{O}(B^2)$ time as detailed in Section D. The resulting overall time complexity is $\mathcal{O}(I_{\text{solver}} B^2)$, where I_{solver} represents the total number of iterations (we restrict $I_{\text{solver}} \leq 50$). Crucially, the solver relies only on **element-wise operations** and **matrix-vector multiplications**, making it highly efficient on GPUs. Full mathematical and implementation details are provided in Section D.

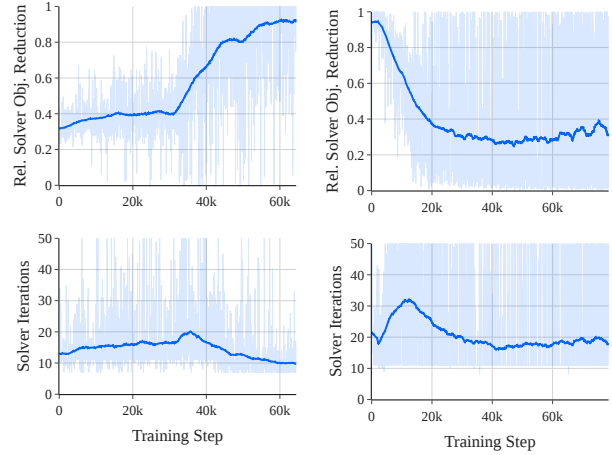
Experiment: Evaluating solver efficiency. We evaluate the performance of our solver in terms of convergence speed and solution quality. We quantify quality by computing the *relative solver objective reduction*, defined as $(H(\mathbf{q}) - H(-\mathbf{Q}_X \mathbf{w}^* + \mathbf{q})) / H(\mathbf{q})$, where $\mathbf{Q}_X$ corresponds to the corresponding matrix $\mathbf{Q}_L$ for SGD (Equation (10)) or $\mathbf{Q}_{1,L}$ for AdamW (Equation (14)). We remark that $H(\mathbf{q})$ and $H(-\mathbf{Q}_X \mathbf{w}^* + \mathbf{q})$ represent the interference generated by the standard update $H(-\mathbf{Q}_X \mathbf{w}^* + \mathbf{q})$ and our method, respectively. Hence, the numerator represents the interference avoided by using our solver’s output $\mathbf{w}^*$. As illustrated in Figure 2, the solver consistently converges within a small

³In practice, we perform a few warm-up steps of Projected Gradient Descent to quickly provide a good starting point for CP.

Algorithm 1 Proposed Training Iteration for SGD

- 1: **Input:** Batch $\mathbf{x}$
- 2: $\mathbf{x}_L \leftarrow F_{1:L-1}(\mathbf{x}; \boldsymbol{\theta}_{1:L-1})$ {Backbone}
- 3: $\mathbf{y}_L \leftarrow F_L(\mathbf{x}_L; \boldsymbol{\theta}_L)$ {Final layer}
- 4: $\mathcal{L} \leftarrow \mathcal{L}(\mathbf{y}_L)$
- 5: **Compute** $\mathbf{Q}_L, \mathbf{G}_L \Delta \boldsymbol{\theta}_{b,L}, \|\Delta \boldsymbol{\theta}_{b,L}\|$ {Sections 4.5, B, C}
- 6: **Solve Equation (10) for** $\mathbf{w}^*$ {Algorithm 7}
- 7: $\mathcal{L}_{\mathbf{w}^*} \leftarrow \sum_{i=1}^B (1 + [\mathbf{w}^*]_i) \mathcal{L}_i$
- 8: Compute gradients of $\mathcal{L}_{\mathbf{w}^*}$ and execute an SGD step

number of iterations while securing substantial objective reductions. Additional plots are provided in Section G.



(a) ResNet-44, CIFAR-10

(b) ViT-c, CIFAR-100

Figure 2. Solver convergence behavior. **Top Row:** Relative objective reduction achieved by the solver (higher is better). **Bottom Row:** Number of iterations required to reach convergence.

4.7. Full Training Algorithm and Computational Complexity

Algorithm 1 details the full proposed training iteration for SGD. The network is decomposed as $F(\mathbf{x}; \boldsymbol{\theta}) = F_L(F_{1:L-1}(\mathbf{x}; \boldsymbol{\theta}_{1:L-1}); \boldsymbol{\theta}_L)$, where F_L represents a standard linear layer (whose output $\mathbf{y}_L$ corresponds to the logits in classification tasks). The main modifications to a standard training loop are Steps 5 and 6 (highlighted in red), which efficiently formulate and solve our surrogate optimization problem. Computing the weighted loss in Step 7 is the final difference, though its computational cost is negligible.

A practical consequence of using final-layer approximations is that these estimated statistics may occasionally diverge from their true global values, posing optimization risks. To prevent instabilities, we apply two **lightweight safeguards** (with time complexity $\mathcal{O}(B^2)$) before invoking the solver. Specifically, we maintain exponential moving averages (EMA) for $\|\Delta \boldsymbol{\theta}_{b,L}\|^2$ and the condition number of $\mathbf{Q}_L$.

If the current step’s $\|\Delta\theta_{b,L}\|^2$ spikes abnormally compared to its EMA, we clip it. On the other hand, if $\mathbf{Q}_L$ becomes ill-conditioned relative to its EMA condition number, we apply adaptive identity preconditioning. In Section C we provide the exact pseudocode and further details.

Extending Algorithm 1 to AdamW is straightforward; the difference is that step 5 involves solving Equation (14), which requires modifying the preceding step to compute the matrices $\mathbf{Q}_{1,L}$ and $\mathbf{Q}_{2,L}$ instead of $\mathbf{Q}_L$.

In Section H, we present a generalized formulation of our method that expands the last-layer approximation to include more layers (i.e. $\ell < L$). Because not all operations between layer ℓ and L are linear, the implicit computation techniques of Section 4.5 cannot be applied. Consequently, for this general form we explicitly compute the per-sample gradients (via PyTorch’s `vmap`). In this section, we also profile the substantial memory and runtime overheads incurred as per-sample gradients are explicitly computed for more layers. Finally, we show that including more layers may introduce numerical imprecisions that degrade overall performance.

Computational Overhead Complexity: The additional computations of our method compared to a standard training loop stem entirely from steps 5 and 6 in Algorithm 1. Assuming the last linear layer has dimensions (d_{in}, d_{out}) , computing the quantities for the optimization problem in step 5 requires $\mathcal{O}(B^2(d_{in} + d_{out}))$ operations for SGD (Section B). For AdamW, due to the additional low-rank SVD requirement, this increases to $\mathcal{O}(rd_{in}d_{out} + rB^2(d_{in} + d_{out}))$. Furthermore, applying our lightweight safeguards for numerical stability (Section C) requires $\mathcal{O}(I_L B^2)$ time, where I_L represents the number of (Lanczos) iterations used to estimate the condition number of $\mathbf{Q}_L$ (or $\mathbf{Q}_{1,L}$). Moving to step 6, the solver operates in $\mathcal{O}(I_{solver} B^2)$ time, iterating at most I_{solver} times (Section D).

Because we upper-bound the maximum allowed rank r and the maximum iterations I_L and I_{solver} , these terms behave as constants. Therefore, the total asymptotic complexity of the overhead reduces to $\mathcal{O}(B^2(d_{in} + d_{out}))$ for SGD and $\mathcal{O}(d_{in}d_{out} + B^2(d_{in} + d_{out}))$ for AdamW. Notice that this extra computation *depends exclusively on the batch size and the dimensions of the final layer*. Consequently, as **the input dimensions or the backbone architecture scales up**—a common reality for modern, huge models where the backbone massively dominates total compute—**this overhead remains fixed**. Finally, we observed completely negligible memory overheads in our benchmarks. The only exception was a minor $\sim 2\%$ increase on the main GPU when training ConvNeXt-Tiny on ImageNet under a 4-GPU Distributed Data Parallel (DDP) setup, resulting from the need to gather and compute the aggregate optimization quantities (e.g., $\mathbf{Q}$ and $\mathbf{G}\Delta\theta_b$) over the collective global batch size.

5. Experiments

We evaluate our method on the downstream task of image classification to demonstrate improved test accuracy with minimal memory and runtime overhead.

5.1. Architectures, Datasets, and Training Details

We evaluate performance on CIFAR-10, CIFAR-100, Tiny-ImageNet, and ImageNet-100. We consider both convolutional and transformer architectures. For ResNets on CIFAR, we follow (He et al., 2016) but substitute Batch Normalization (Ioffe & Szegedy, 2015) with Group Normalization (Wu & He, 2018), as PyTorch’s `vmap`—required for computing explicit per-sample gradients $\mathbf{G}$ in certain experiments (e.g., Figure 1)—lacks BatchNorm support. To avoid memory overflow when needing to compute the full $\mathbf{G}$, we employ ResNet-20 and a small Vision Transformer (denoted ViT-small). Otherwise, we use ResNet-44 and the ViT implementation from (omihub777, 2021). For larger datasets (Tiny-ImageNet, ImageNet-100), we employ ResNet-50 and ConvNeXt-Tiny (Liu et al., 2022). Note that ResNet is trained using SGD, while ViT and ConvNeXt employ AdamW. Further details are provided in Section E.

Across all experiments, we fix the trust-region parameter $c = 0.2$. Together with the constraint $\mathbf{w} \leq 2$, these form the main algorithmic choices of our method. While more parameters are required, those are auxiliary, serving to ensure numerical stability when estimating the defining quantities and to ensure solver efficiency. Fundamentally, as the underlying problem is convex, provided it is robustly defined and the solver swiftly converges to a near-optimal point, the general behavior of our method is expected to depend little on exactly how these auxiliary parameters are set.

5.2. Image Classification Performance

Table 1 reports test accuracy and relative time overhead across varying batch sizes. Our approach consistently improves upon standard training, with particularly pronounced gains on CIFAR-100.

Intuitively, *larger batch sizes* include more samples, increasing the probability of conflicting gradients. This presents a *greater opportunity for our method to mitigate conflicts*, potentially yielding higher gains. Indeed, as B scales from 128 to 1024, the absolute accuracy improvement over the baseline steadily increases: for ResNet-44 on CIFAR-100 (+0.25%, +0.57%, +0.86%, +0.88%), for ViT on CIFAR-10 (+0.49%, +0.34%, +0.58%, +0.70%), and for ViT on CIFAR-100 (+0.58%, +0.79%, +1.38%, +2.24%).

An exception occurs with ResNet-44 on CIFAR-10, where gains degrade (+0.22%, +0.26%, +0.15%, -0.22%). The main reason for the performance drop at $B = 1024$ is that

Table 1. Test accuracy (%) and relative time overhead (expressed as a multiplication factor) for ResNet-44 and ViT on CIFAR-10/100 across varying batch sizes (BS).

BS	CIFAR-10			CIFAR-100		
	BASE	OURS	TIME	BASE	OURS	TIME
<i>ResNet-44</i>						
128	91.71	91.93	1.13×	65.22	65.50	1.30×
256	91.22	91.48	1.07×	64.84	65.41	1.11×
512	90.60	90.75	1.03×	63.51	64.37	1.05×
1024	89.22	89.00	1.03×	61.43	62.31	1.03×
<i>ViT-c</i>						
128	89.94	90.34	1.08×	63.05	63.63	1.10×
256	89.62	89.96	1.02×	62.27	63.06	1.08×
512	89.59	90.17	1.01×	60.73	62.11	1.05×
1024	88.91	89.61	1.01×	58.82	61.06	1.04×

Table 2. Test accuracy (%) and relative time overhead for ResNet-50 and ConvNeXt-T on Tiny-ImageNet and ImageNet-100 across different configurations.

CONFIG	BASE	OURS	TIME
<i>ResNet-50 (Tiny-ImageNet, B = 256)</i>			
GroupNorm	47.94	48.69	1.07×
BatchNorm	56.37	56.20	1.02×
<i>ConvNeXt-T (Tiny-ImageNet, B = 4096)</i>			
w/o Mixup	54.08	54.54	1.18×
with Mixup	59.69	59.78	1.17×
<i>ConvNeXt-T (ImageNet-100, B = 4096)</i>			
w/o Mixup	76.50	76.66	1.03×
with Mixup	78.60	79.76	1.03×

the final linear layer of ResNet-44 has only 640 parameters ($d_{in}=64, d_{out}=10$). Consequently, $\mathbf{G}_L \in \mathbb{R}^{1024 \times 640}$, rendering the Gram matrix $\mathbf{Q}_L = \mathbf{G}_L \mathbf{G}_L^T$ rank-deficient and highly ill-conditioned. This induces instability; in fact, this is the only setting where we had to reduce the trust-region radius to $c=0.1$ to prevent severe performance degradation.⁴

Regarding computational efficiency, represented as a runtime multiplier in Table 1, we observe two key trends. First, transitioning from ResNet-44 (0.6M parameters) to ViT (6.3M parameters) significantly reduces the relative overhead. This aligns with our analysis in Section 4.7: increasing the backbone size increases the time spent in its forward and backward passes, whereas our method’s overhead remains fixed, thereby reducing its relative cost. Second, although standard training scales linearly with B and our method scales quadratically, the relative overhead actually decreases for larger batches. We attribute this to GPU utilization: for the tested batch sizes ($B \leq 1024$), the $\mathcal{O}(B^2)$

⁴In contrast, the ViT has a linear head with 3840 parameters and as $3840 > 1024$ similar singularity issues do not appear.

matrix-vector operations used by our method are still small for modern GPUs. At smaller B , these operations do not fully utilize the available parallelism, so fixed GPU overheads account for a larger fraction of their runtime. As B increases, the same operations use the hardware more efficiently, reducing the relative overhead.

Table 2 presents results for larger models—ResNet-50 (25M parameters) and ConvNeXt-T (29M parameters)—on larger datasets. Since BatchNorm and Mixup/Cutmix break sample independence within a batch, changing the statistics of the required quantities (e.g., $\mathbf{Q}$), we test their effect by evaluating configurations both with and without them. Excluding ResNet-50 with BatchNorm, our approach consistently improves test accuracy, achieving the most substantial gains on ConvNeXt-T trained with Mixup on ImageNet-100.

For ConvNeXt-T, going from Tiny-ImageNet to ImageNet-100 the relative overhead is reduced from 18% to just 3%. This agrees with our analysis in Section 4.7: as the input size increases (here from images 64×64 to 224×224), the backbone processing time increases, but our method’s overhead remains the same, decreasing the relative overhead.

We used a Distributed Data Parallel (DDP) setup ($4 \times T4$ GPUs for Tiny-ImageNet, $4 \times A100$ for ImageNet-100) to train ConvNeXt-T. Standard DDP already requires gradient aggregation across GPUs (PyTorch’s `all_reduce`) and state synchronization (`broadcast`). Our method adds two synchronization steps: i) an `all_gather` to collect the last-layer per-sample gradients from all GPUs and form the global required quantities (e.g., $\mathbf{Q}$), and ii) a `broadcast` to distribute the solver’s computed optimal weights. These extra synchronizations could potentially be removed through a practical relaxation in which each GPU uses its local batch to independently form and solve its own optimization problem (Equations (10) and (14)). This effectively approximates the global $\mathbf{Q}_L$ as block-diagonal, assuming that sample gradients across devices are uncorrelated.

6. Conclusion

We proposed a practical framework for reducing the interference that updates can induce on individual samples. Although the natural formulation is computationally prohibitive, we successfully turn it into an efficient surrogate by moving the optimization to batch space, restricting the required quantities to the final layer and implicitly computing them without materializing per-sample gradients. We finally introduce a GPU-friendly solver to solve the resulting surrogate problem. This yields a lightweight modification of standard optimizers such as SGD with momentum and AdamW. Experiments on image classification benchmarks demonstrate that our method reduces per-sample interference and improves test accuracy with small runtime overhead.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

References

- Abadi, M., Chu, A., Goodfellow, I., McMahan, H. B., Mironov, I., Talwar, K., and Zhang, L. Deep learning with differential privacy. In *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*, pp. 308–318, 2016.
- Bauschke, H. H. and Borwein, J. M. On projection algorithms for solving convex feasibility problems. *SIAM review*, 38(3):367–426, 1996.
- Boyle, J. P. and Dykstra, R. L. A method for finding projections onto the intersection of convex sets in hilbert spaces. In *Advances in Order Restricted Statistical Inference: Proceedings of the Symposium on Order Restricted Statistical Inference held in Iowa City, Iowa, September 11–13, 1985*, pp. 28–47. Springer, 1986.
- Bu, Z., Wang, Y.-X., Zha, S., and Karypis, G. Differentially private optimization on large model at small cost. In *International Conference on Machine Learning*, pp. 3192–3218. PMLR, 2023.
- Chambolle, A. and Pock, T. A first-order primal-dual algorithm for convex problems with applications to imaging. *Journal of mathematical imaging and vision*, 40(1):120–145, 2011.
- Chaudhry, A., Ranzato, M., Rohrbach, M., and Elhoseiny, M. Efficient lifelong learning with a-GEM. In *International Conference on Learning Representations*, 2019. URL https://openreview.net/forum?id=Hkf2_sC5FX.
- Chen, X., Liang, C., Huang, D., Real, E., Wang, K., Pham, H., Dong, X., Luong, T., Hsieh, C.-J., Lu, Y., et al. Symbolic discovery of optimization algorithms. *Advances in neural information processing systems*, 36:49205–49233, 2023.
- Combettes, P. L. and Pesquet, J.-C. Proximal splitting methods in signal processing. In *Fixed-point algorithms for inverse problems in science and engineering*, pp. 185–212. Springer, 2011.
- Dangel, F., Kunstner, F., and Hennig, P. Backpack: Packing more into backprop. In *International Conference on Learning Representations*, 2020. URL <https://openreview.net/forum?id=BJlrF24twB>.
- Dong, X., Wu, R., Xiong, C., Li, H., Cheng, L., He, Y., Qian, S., Cao, J., and Mo, L. Gdod: Effective gradient descent using orthogonal decomposition for multi-task learning. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*, pp. 386–395, 2022.
- Farajtabar, M., Azizan, N., Mott, A., and Li, A. Orthogonal gradient descent for continual learning. In *International conference on artificial intelligence and statistics*, pp. 3762–3773. PMLR, 2020.
- He, K., Zhang, X., Ren, S., and Sun, J. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 770–778, 2016.
- Hoffer, E., Hubara, I., and Soudry, D. Train longer, generalize better: closing the generalization gap in large batch training of neural networks. *Advances in neural information processing systems*, 30, 2017.
- Hu, G., Zhang, W., Ding, H., and Zhu, W. Gradient episodic memory with a soft constraint for continual learning. *arXiv preprint arXiv:2011.07801*, 2020.
- Ioffe, S. and Szegedy, C. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International conference on machine learning*, pp. 448–456. pmlr, 2015.
- Jordan, K., Jin, Y., Boza, V., Jiacheng, Y., Cecista, F., Newhouse, L., and Bernstein, J. Muon: An optimizer for hidden layers in neural networks, 2024. URL <https://kellerjordan.github.io/posts/muon>, 6, 2024.
- Kingma, D. P. and Ba, J. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- Li, X., Tramer, F., Liang, P., and Hashimoto, T. Large language models can be strong differentially private learners. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=bVuP3ltATMz>.
- Liu, B., Liu, X., Jin, X., Stone, P., and Liu, Q. Conflict-averse gradient descent for multi-task learning. *Advances in Neural Information Processing Systems*, 34:18878–18890, 2021a.
- Liu, L., Jiang, H., He, P., Chen, W., Liu, X., Gao, J., and Han, J. On the variance of the adaptive learning rate and beyond. In *International Conference on Learning Representations*, 2020. URL <https://openreview.net/forum?id=rkgz2aEKDr>.

- Liu, L., Li, Y., Kuang, Z., Xue, J.-H., Chen, Y., Yang, W., Liao, Q., and Zhang, W. Towards impartial multi-task learning. In *International conference on learning representations*, 2021b.
- Liu, Z., Mao, H., Wu, C.-Y., Feichtenhofer, C., Darrell, T., and Xie, S. A convnet for the 2020s. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 11976–11986, June 2022.
- Lopez-Paz, D. and Ranzato, M. Gradient episodic memory for continual learning. *Advances in neural information processing systems*, 30, 2017.
- Loshchilov, I. and Hutter, F. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=Bkg6RiCqY7>.
- Luo, Y., Yang, Z., Meng, F., Li, Y., Zhou, J., and Zhang, Y. An empirical study of catastrophic forgetting in large language models during continual fine-tuning. *IEEE Transactions on Audio, Speech and Language Processing*, 2025.
- Navon, A., Shamsian, A., Achituve, I., Maron, H., Kawaguchi, K., Chechik, G., and Fetaya, E. Multi-task learning as a bargaining game. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pp. 16428–16446. PMLR, 2022. URL <https://proceedings.mlr.press/v162/navon22a.html>.
- Nesterov, Y. A method for solving the convex programming problem with convergence rate $o(1/k^2)$. *Doklady Akademii Nauk SSSR*, 269:543–547, 1983.
- omihub777. Vit-cifar. <https://github.com/omihub777/ViT-CIFAR>, 2021. Accessed: 2026-01-24.
- Rochette, G., Manoel, A., and Tramel, E. W. Efficient per-example gradient computations in convolutional neural networks. *ArXiv*, abs/1912.06015, 2019. URL <https://api.semanticscholar.org/CorpusID:209324041>.
- Roulet, V. and Agarwala, A. Per-example gradients: a new frontier for understanding and improving optimizers. *arXiv preprint arXiv:2510.00236*, 2025.
- Saha, G., Garg, I., and Roy, K. Gradient projection memory for continual learning. In *International Conference on Learning Representations*, 2021. URL <https://openreview.net/forum?id=3AOj0RCNC2>.
- Schölkopf, B., Herbrich, R., and Smola, A. J. A generalized representer theorem. In *International conference on computational learning theory*, pp. 416–426. Springer, 2001.
- Senushkin, D., Patakin, N., Kuznetsov, A., and Konushin, A. Independent component alignment for multi-task learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 20083–20093, 2023.
- Shazeer, N. and Stern, M. Adafactor: Adaptive learning rates with sublinear memory cost. In *International Conference on Machine Learning*, pp. 4596–4604. PMLR, 2018.
- Sutskever, I., Martens, J., Dahl, G., and Hinton, G. On the importance of initialization and momentum in deep learning. *Proceedings of the 30th International Conference on Machine Learning*, 2013.
- Wu, Y. and He, K. Group normalization. In *Proceedings of the European conference on computer vision (ECCV)*, pp. 3–19, 2018.
- Yang, E., Pan, J., Wang, X., Yu, H., Shen, L., Chen, X., Xiao, L., Jiang, J., and Guo, G. Adatask: A task-aware adaptive learning rate approach to multi-task learning. In *Proceedings of the AAAI conference on artificial intelligence*, volume 37, pp. 10745–10753, 2023.
- Yin, D., Pananjady, A., Lam, M., Papailiopoulos, D., Ramchandran, K., and Bartlett, P. Gradient diversity: a key ingredient for scalable distributed learning. In *International Conference on Artificial Intelligence and Statistics*, pp. 1998–2007. PMLR, 2018.
- You, Y., Li, J., Reddi, S., Hseu, J., Kumar, S., Bhojanapalli, S., Song, X., Demmel, J., Keutzer, K., and Hsieh, C.-J. Large batch optimization for deep learning: Training bert in 76 minutes. In *International Conference on Learning Representations*, 2020. URL <https://openreview.net/forum?id=Syx4wnEtvH>.
- Yousefpour, A., Shilov, I., Sablayrolles, A., Testuggine, D., Prasad, K., Malek, M., Nguyen, J., Ghosh, S., Bharadwaj, A., Zhao, J., et al. Opacus: User-friendly differential privacy library in pytorch. *arXiv preprint arXiv:2109.12298*, 2021.
- Yu, T., Kumar, S., Gupta, A., Levine, S., Hausman, K., and Finn, C. Gradient surgery for multi-task learning. *Advances in neural information processing systems*, 33: 5824–5836, 2020.
- Zhai, X., Kolesnikov, A., Houlsby, N., and Beyer, L. Scaling vision transformers. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 12104–12113, 2022.

A. Proof of Proposition 4.1

Defining the displacement vector $\delta := \Delta\theta - \Delta\theta_b \in \mathbb{R}^\Theta$, $\mathbf{s} := \mathbf{G}\Delta\theta_b \in \mathbb{R}^B$, the radius $\rho := c\|\Delta\theta_b\|$, and using the definition $H(\mathbf{v}) := \sum_{i=1}^B \max(0, [\mathbf{v}]_i)$, we can directly rewrite the formulation (3) as:

$$\begin{aligned} \min_{\delta} \quad & J(\delta) := H(\mathbf{G}\delta + \mathbf{s}) \\ \text{s.t.} \quad & \|\delta\| \leq \rho. \end{aligned} \tag{16}$$

Part 1: Existence of a solution in $\text{Range}(\mathbf{G}^\top)$. Any vector $\delta \in \mathbb{R}^\Theta$ can be uniquely decomposed into two orthogonal components:

$$\delta = \delta_{\parallel} + \delta_{\perp},$$

where $\delta_{\parallel} \in \text{Range}(\mathbf{G}^\top)$ (i.e., $\delta_{\parallel} = \mathbf{G}^\top \mathbf{v}$ for some $\mathbf{v} \in \mathbb{R}^B$) and $\delta_{\perp} \in \text{Null}(\mathbf{G})$ (i.e., $\mathbf{G}\delta_{\perp} = \mathbf{0}$). Since $\mathbf{G}\delta = \mathbf{G}\delta_{\parallel}$, we have $J(\delta) = J(\delta_{\parallel})$. Moreover, by orthogonality,

$$\|\delta\|^2 = \|\delta_{\parallel}\|^2 + \|\delta_{\perp}\|^2 \Rightarrow \|\delta_{\parallel}\| \leq \|\delta\|.$$

Therefore, if δ is feasible (i.e., $\|\delta\| \leq \rho$), then $\delta_{\parallel}$ is also feasible and achieves the same objective value. Hence, there always exists an optimal solution lying in $\text{Range}(\mathbf{G}^\top)$.

Part 2: Non-positivity of coefficients ($\mathbf{w} \leq \mathbf{0}$). Let $\mathcal{S}^*$ denote the set of optimal solutions to Equation (16). We aim to prove that the minimum-norm optimal solution $\delta^\dagger \in \mathcal{S}^*$ satisfies $\delta^\dagger = \mathbf{G}^\top \mathbf{w}$ with $\mathbf{w} \leq \mathbf{0}$ (recall that $\Delta\theta^* = \Delta\theta_b + \delta^\dagger$). To this end, we add a small L_2 regularization term and consider the modified problem:

$$\min_{\delta} J(\delta) + \epsilon \|\delta\|^2 \quad \text{s.t.} \quad \|\delta\|^2 \leq \rho^2,$$

where $\epsilon > 0$ is arbitrarily small. The objective is now strictly convex and therefore has a unique minimizer δ_ϵ . The subdifferential of $J(\delta) = H(\mathbf{G}\delta + \mathbf{s})$ is given by:

$$\partial J(\delta) = \left\{ \mathbf{G}^\top \gamma \mid [\gamma]_i = \begin{cases} 1, & \text{for } [\mathbf{G}\delta + \mathbf{s}]_i > 0, \\ \in [0, 1], & \text{for } [\mathbf{G}\delta + \mathbf{s}]_i = 0, \\ 0, & \text{for } [\mathbf{G}\delta + \mathbf{s}]_i < 0 \end{cases} \right\}.$$

We remark that every admissible γ satisfies $\gamma \in [0, 1]^B$. By the KKT stationarity condition, $\mathbf{0} \in \partial J(\delta_\epsilon) + 2(\epsilon + \lambda)\delta_\epsilon$, where $\lambda \geq 0$ is the Lagrange multiplier associated with the constraint $\|\delta\|^2 \leq \rho^2$. Hence, there exists some $\gamma \geq \mathbf{0}$ such that

$$\mathbf{G}^\top \gamma + 2(\epsilon + \lambda)\delta_\epsilon = \mathbf{0} \Rightarrow \delta_\epsilon = \mathbf{G}^\top \left(-\frac{\gamma}{2\epsilon + 2\lambda} \right).$$

Defining $\mathbf{w}_\epsilon := -\frac{\gamma}{2(\epsilon + \lambda)}$, we obtain $\mathbf{w}_\epsilon \leq \mathbf{0}$ since $\gamma \geq \mathbf{0}$ and $\epsilon + \lambda > 0$. Finally, as $\epsilon \rightarrow 0$, the sequence δ_ϵ converges to the minimum-norm optimal solution $\delta^\dagger \in \mathcal{S}^*$ (since if two solutions are optimal, and so they have the same J , the one with the smaller norm will have a smaller $J(\delta) + \epsilon\|\delta\|^2$). Therefore, $\delta^\dagger = \mathbf{G}^\top \mathbf{w}$ for some $\mathbf{w} \leq \mathbf{0}$.

Part 3: Uniqueness of minimum distance. Consider any optimal solution $\Delta\theta \in \mathcal{S}^*$ with displacement $\delta = \Delta\theta - \Delta\theta_b$, and decompose $\delta = \delta_{\parallel} + \delta_{\perp}$ as above. From Part 1 we know that $\delta' = \delta_{\parallel}$ is also optimal. Since

$$\|\delta\|^2 = \|\delta_{\parallel}\|^2 + \|\delta_{\perp}\|^2 \geq \|\delta_{\parallel}\|^2 = \|\delta'\|^2$$

any solution with $\delta_{\perp} \neq \mathbf{0}$ has strictly larger distance to $\Delta\theta_b$ than the corresponding solution δ' with $\delta_{\perp} = \mathbf{0}$. Hence, among all optimal solutions, the unique minimizer of $\|\Delta\theta - \Delta\theta_b\|$ is the one with $\delta \in \text{Range}(\mathbf{G}^\top)$, i.e., of the form $\Delta\theta_b + \mathbf{G}^\top \mathbf{w}$.

Connection to the Representer Theorem (Schölkopf et al., 2001). A closely related statement follows by considering a Lagrangian relaxation of the ball constraint, yielding a regularized objective of the form

$$\min_{\delta} H(\mathbf{G}\delta + \mathbf{s}) + \lambda \|\delta\|^2,$$

for some $\lambda > 0$. The Representer Theorem then directly guarantees that the optimal δ lies in the span of the data representer $\{\mathbf{G}_i\}_{i=1}^B$. This is consistent with Part 1. Our argument additionally characterizes the sign of the coefficients in the representation, i.e., $\mathbf{w} \leq \mathbf{0}$.

B. Computing the Defining Quantities of the Optimization Problem Without Materializing Per-Sample Gradients

In this section, we study the case of a single linear layer. Because our focus here is not limited to the final layer, we drop the subscript L —used in the main text to denote the last linear layer—for clarity. Consider a linear layer with parameters $\theta_w \in \mathbb{R}^{d_{out} \times d_{in}}$ and bias $\theta_b \in \mathbb{R}^{d_{out}}$, receiving inputs $\mathbf{x} \in \mathbb{R}^{B \times d_{in}}$ and producing outputs $\mathbf{y} \in \mathbb{R}^{B \times d_{out}}$ such that $[\mathbf{y}]_i = [\mathbf{x}\theta_w]_i + \theta_b, \forall i \in \{1, 2, \dots, B\}$. With $[\mathbf{A}]_i$ we refer to the i -th element if $\mathbf{A}$ is a vector and the i -th row if it is a matrix. Let $\mathbf{g}_y = \nabla_y \mathcal{L} \in \mathbb{R}^{B \times d_{out}}$ denote the gradient of the loss with respect to these outputs. The per-sample gradient with respect to the weight parameters is $\mathbf{G}_w = \mathbf{g}_y \otimes \mathbf{x} \in \mathbb{R}^{B \times (d_{out} d_{in})}$, where $\otimes$ denotes the Khatri-Rao (i.e., row-wise Kronecker) product, meaning $[\mathbf{G}_w]_i = [\mathbf{g}_y]_i \otimes [\mathbf{x}]_i$ gives the flattened gradient for the i -th sample⁵. The per-sample gradients with respect to the bias parameters are simply $\mathbf{G}_b = \mathbf{g}_y \in \mathbb{R}^{B \times d_{out}}$.

B.1. Standard form $\mathbf{Q} = \mathbf{G}\mathbf{G}^\top$ (SGD)

Leveraging the properties of the Khatri-Rao product, the total Gram matrix $\mathbf{Q} \in \mathbb{R}^{B \times B}$ can be factorized as:

$$\mathbf{Q} = \mathbf{G}_w \mathbf{G}_w^\top + \mathbf{G}_b \mathbf{G}_b^\top = (\mathbf{x}\mathbf{x}^\top) \odot (\mathbf{g}_y \mathbf{g}_y^\top) + (\mathbf{g}_y \mathbf{g}_y^\top) = (\mathbf{x}\mathbf{x}^\top + \mathbf{I}) \odot (\mathbf{g}_y \mathbf{g}_y^\top)$$

where $\mathbf{I}$ is the $B \times B$ identity matrix. This factorization allows us to compute $\mathbf{Q}$ as $(\mathbf{x}\mathbf{x}^\top + \mathbf{I}) \odot (\mathbf{g}_y \mathbf{g}_y^\top)$ instead of $\mathbf{G}_w \mathbf{G}_w^\top + \mathbf{G}_b \mathbf{G}_b^\top$. Therefore it eliminates the need to materialize the per-sample weight gradients $\mathbf{G}_w \in \mathbb{R}^{B \times (d_{out} d_{in})}$, circumventing a prohibitive $\mathcal{O}(B d_{in} d_{out})$ memory cost. The time complexity is reduced from $\mathcal{O}(B^2 d_{in} d_{out})$ to $\mathcal{O}(B^2 (d_{in} + d_{out}))$. In Pseudocode 1, we provide a minimal example demonstrating how the Khatri-Rao trick can be used for the computation of the Gram matrix $\mathbf{Q}$ compared to the standard baseline.

Pseudocode 1. Khatri-Rao Trick vs. Baseline in PyTorch

```
# Dimensions: x [B, d_in], grad_y [B, d_out]

# --- Baseline ---
per_sample_grad_w = grad_y.unsqueeze(2) * x.unsqueeze(1) # [B, d_out, d_in]
grad_w_flat = per_sample_grad_w.flatten(start_dim=1) # [B, d_out * d_in]
Q_bias = grad_y @ grad_y.t()
Q_weight = grad_w_flat @ grad_w_flat.t()
Q_baseline = Q_weight + Q_bias

# --- Khatri-Rao Trick ---
Q_bias = grad_y @ grad_y.t()
Q_optimized = (x @ x.t() + torch.eye(B)) * Q_bias
```

In Table 3, each row corresponds to one batch size. Entries are reported in the format: *time (ms)*, *memory (MB)* for Baseline, and *time (improvement)*, *memory (improvement)* for Khatri-Rao, where improvements are percentages relative to Baseline. For example, the memory improvement is defined as $100 \cdot \frac{Mem.Baseline - Mem.Khatri - Rao}{Mem.Baseline}$. We emphasize that the *maximum improvement is 100%*. An NVIDIA Tesla T4 GPU with 16 GB memory was used. Note that Tables (a)-(b) correspond to layer dimensions used for *ResNet-44-gn* architectures for CIFAR-10 and CIFAR-100 respectively. The tables show that the gains are an order of magnitude both in time and in memory, especially as the dimensions are increased.

⁵Note the Kronecker product for one dimensional vectors coincides to their flattened outer product.

Reducing Per-Sample Interference in Stochastic Optimization

(a) $d_{in} = 64, d_{out} = 10$			(b) $d_{in} = 64, d_{out} = 100$		
BS	Baseline (ms, MB)	Khatri-Rao (ms, MB)	BS	Baseline (ms, MB)	Khatri-Rao (ms, MB)
128	0.14, 8.75	0.107 (25%), 8.41 (4%)	128	0.22, 11.9	0.10 (54%), 8.46 (29%)
512	0.42, 13.6	0.343 (19%), 12.3 (10%)	512	1.63, 26.2	0.35 (79%), 12.5 (53%)
2048	2.60, 79.3	1.07 (59%), 72.7 (8%)	2048	15.2, 128	1.03 (93%), 73.4 (43%)
(c) $d_{in} = 2048, d_{out} = 100$			(d) $d_{in} = 2048, d_{out} = 1000$		
BS	Baseline (ms, MB)	Khatri-Rao (ms, MB)	BS	Baseline (ms, MB)	Khatri-Rao (ms, MB)
256	9.08, 231	0.185 (98%), 11.2 (95%)	256	127, 2212	0.326 (100%), 12.1 (100%)
1024	155, 913	1.95 (99%), 32.5 (96%)	1024	Out-of-Memory	2.44, 36.1
4096	3381, 3818	22.3 (99%), 298 (92%)	4096	Out-of-Memory	29.9, 312

Table 3. Benchmark comparison of Baseline and Khatri-Rao across batch sizes and layer dimensions.

B.2. Weighted form $\mathbf{Q} = \mathbf{G} \text{diag}(\boldsymbol{\tau}) \mathbf{G}^\top$ (AdamW)

Let $[\mathbf{M}]_{i,j}$ denote the element in the i -th row and j -th column of a matrix $\mathbf{M}$. We assume $\boldsymbol{\tau} \in \mathbb{R}^{d_{out} \times d_{in}}$ and with $\text{diag}(\boldsymbol{\tau})$ we mean that first we flatten $\boldsymbol{\tau}$ in the same way that each row (corresponding to each sample of the batch) of $\mathbf{G}_w$ was flattened, and then we diagonalize the resulting vector. Previously we could directly use the Khatri-Rao trick as we could write:

$$[\mathbf{G}_w \mathbf{G}_w^\top]_{i,j} = \sum_{q,p} [\mathbf{x}]_{i,q} [\mathbf{x}]_{j,q} [\mathbf{g}_y]_{i,p} [\mathbf{g}_y]_{j,p} = \left(\sum_q [\mathbf{x}]_{i,q} [\mathbf{x}]_{j,q} \right) \left(\sum_p [\mathbf{g}_y]_{i,p} [\mathbf{g}_y]_{j,p} \right).$$

Unfortunately, the existence of the diagonal matrix complicates the formula as:

$$[\mathbf{G}_w \text{diag}(\boldsymbol{\tau}) \mathbf{G}_w^\top]_{i,j} = \sum_{q,p} \tau_{p,q} [\mathbf{x}]_{i,q} [\mathbf{x}]_{j,q} [\mathbf{g}_y]_{i,p} [\mathbf{g}_y]_{j,p},$$

and the $\tau_{p,q}$ inhibits the same factorization. However, if we manage to factorize $\boldsymbol{\tau} = \mathbf{A} \mathbf{V}^\top$, then because $\tau_{p,q} = \sum_n [\mathbf{A}]_{p,n} [\mathbf{V}]_{q,n}$, we can write:

$$[\mathbf{G}_w \text{diag}(\boldsymbol{\tau}) \mathbf{G}_w^\top]_{i,j} = \sum_n \left(\left(\sum_q [\mathbf{V}]_{q,n} [\mathbf{x}]_{i,q} [\mathbf{x}]_{j,q} \right) \left(\sum_p [\mathbf{A}]_{p,n} [\mathbf{g}_y]_{i,p} [\mathbf{g}_y]_{j,p} \right) \right).$$

Such a factorization could be achieved with SVD, setting $\boldsymbol{\tau} = \mathbf{U} \mathbf{S} \mathbf{V}^\top$ with $\mathbf{U} \in \mathbb{R}^{d_{out} \times d_{out}}$, $\mathbf{V} \in \mathbb{R}^{d_{in} \times d_{in}}$ unitary matrices and $\mathbf{S} = \text{diag}(\sigma_1, \dots, \sigma_{\min(d_{in}, d_{out})}) \in \mathbb{R}^{d_{out} \times d_{in}}$ with $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$ the singular values. We choose the SVD factorization because the matrix $\boldsymbol{\tau}^r$ that is closest to $\boldsymbol{\tau}$ in Frobenius norm, i.e., minimizing $\|\boldsymbol{\tau} - \boldsymbol{\tau}^r\|_F$, and can be factorized as $\boldsymbol{\tau}^r = \mathbf{A}^r \mathbf{V}^{r\top}$ with $\mathbf{A}^r \in \mathbb{R}^{d_{out} \times r}$ and $\mathbf{V}^r \in \mathbb{R}^{d_{in} \times r}$, is given by $\mathbf{A} = \mathbf{U}^r \text{diag}(\sigma_1, \dots, \sigma_r)$, where $\mathbf{U}^r \in \mathbb{R}^{d_{out} \times r}$ is the first r columns of $\mathbf{U}$ and $\mathbf{V}^r$ is the first r columns of $\mathbf{V}$. Therefore, given r , we know we have the best possible approximation and we can write:

$$\begin{aligned} [\mathbf{G}_w \text{diag}(\boldsymbol{\tau}) \mathbf{G}_w^\top]_{i,j} &\approx \sum_{k=1}^r \sigma_k \left(\left(\sum_q [\mathbf{V}]_{q,k} [\mathbf{x}]_{i,q} [\mathbf{x}]_{j,q} \right) \left(\sum_p [\mathbf{U}]_{p,k} [\mathbf{g}_y]_{i,p} [\mathbf{g}_y]_{j,p} \right) \right) \\ &= \sum_{k=1}^r \sigma_k (\mathbf{x} \text{diag}(\mathbf{v}_k) \mathbf{x}^\top) \odot (\mathbf{g}_y \text{diag}(\mathbf{u}_k) \mathbf{g}_y^\top) \end{aligned}$$

denoting by $\mathbf{u}_k, \mathbf{v}_k$ the k -th column of $\mathbf{U}^r$ and $\mathbf{V}^r$ respectively.

Similar to before, this factorization bypasses the prohibitive $\mathcal{O}(B d_{in} d_{out})$ memory cost associated with materializing per-sample gradients. In terms of computation, while the baseline requires $\mathcal{O}(B^2 d_{in} d_{out})$ operations to compute the exact weighted Gram matrix, our proposed method reduces the overall time complexity to $\mathcal{O}(r d_{in} d_{out} + r B^2 (d_{in} + d_{out}))$. The first term accounts for the cost of computing a low-rank SVD on $\boldsymbol{\tau}$ (e.g., using `torch.pca_lowrank`). Although performing a full SVD (e.g., using `torch.linalg.svd`) results in higher time complexity $\mathcal{O}(\min(d_{in}, d_{out}) d_{in} d_{out})$, for the matrix dimensions tested in our experiments there was no substantial overhead, so we keep the full-SVD implementation.

Pseudocode 2. Khatri-Rao Trick with SVD vs. Baseline in PyTorch (AdamW)

```
# Dimensions: x [B, d_in], grad_y [B, d_out], tau_w [d_out, d_in], tau_b [d_out]

# --- Baseline ---
per_sample_grad_w = grad_y.unsqueeze(2) * x.unsqueeze(1) # [B, d_out, d_in]
grad_w_flat = per_sample_grad_w.flatten(start_dim=1) # [B, d_out * d_in]
Q_weight = (grad_w_flat * tau_w.unsqueeze(0)) @ grad_w_flat.t()
Q_bias = (grad_y * tau_b.unsqueeze(0)) @ grad_y.t()
Q_baseline = Q_weight + Q_bias

# --- Khatri-Rao Trick with SVD ---
U, S, Vh = torch.linalg.svd(tau_w, full_matrices=False)
u = U[:, :r].t().unsqueeze(1) # [r, 1, d_out]
s = S[:r].view(r, 1, 1) # [r, 1, 1]
v = Vh[:r, :].unsqueeze(1) # [r, 1, d_in]

Gx = (x * v) @ x.t() # [r, B, B]
Gy = (grad_y * u) @ grad_y.t() # [r, B, B]

Q_weight = (s * Gx * Gy).sum(dim=0) # Sum over rank -> [B, B]
Q_bias = (grad_y * tau_b.unsqueeze(0)) @ grad_y.t()
Q_optimized = Q_weight + Q_bias
```

In Table 4, we summarize the benchmark execution. The setup mirrors the one used for the standard form (Section B.1). We used rank-5 approximation ($r = 5$), as is the one used throughout the paper. Table (a) and (b) match the dimensions of ViT-c for CIFAR-10 and CIFAR-100 respectively. Table (c) matches the dimensions of ConvNeXt-Tiny on the Tiny-ImageNet dataset. SVD computations introduce non-negligible overhead that can sometimes increase execution time and memory consumption compared to the baseline; we highlight this performance deterioration in red. However, as layer dimensions progressively scale, the asymptotic benefits of the decomposition prevail, yielding substantial speedups, massive memory reductions and avoiding Out-of-Memory errors. Finally, we see that as batch-size increases memory gains become worse, but the speed gains improve.

(a) $d_{in} = 384, d_{out} = 10$			(b) $d_{in} = 384, d_{out} = 100$		
BS	Baseline (ms, MB)	Khatri-Rao SVD (ms, MB)	BS	Baseline (ms, MB)	Khatri-Rao SVD (ms, MB)
128	0.37, 17.7	1.77 (- 384%), 16.9 (5%)	128	1.34, 51.6	4.77 (- 257%), 17.5 (66%)
512	1.20, 32.2	2.17 (- 81%), 34.4 (- 7%)	512	7.22, 169	6.01 (17%), 34.9 (79%)
2048	12.5, 112	8.40 (33%), 337 (- 201%)	2048	113, 647	14.0 (88%), 332 (49%)
(c) $d_{in} = 768, d_{out} = 200$			(d) $d_{in} = 768, d_{out} = 1000$		
BS	Baseline (ms, MB)	Khatri-Rao SVD (ms, MB)	BS	Baseline (ms, MB)	Khatri-Rao SVD (ms, MB)
256	9.03, 316	9.71 (- 8%), 21.5 (93%)	256	62.3, 1519	89.2 (- 43%), 34.3 (98%)
1024	137, 1222	14.8 (89%), 93.2 (92%)	1024	830, 6027	98.4 (88%), 103 (98%)
4096	2882, 4966	73.8 (97%), 1305 (74%)	4096	Out-of-Memory	212, 1324

Table 4. Benchmark comparison of Baseline and Khatri-Rao with rank-5 SVD approximation (AdamW form).

B.3. Implicit computation of $\Delta\theta_b$ and $G\Delta\theta_b$

To fully realize the computational and memory benefits described above, it is crucial that *all* quantities defining the optimization problem are computed without explicitly instantiating the per-sample gradients $\mathbf{G}$. Evidently, it is pointless to avoid materializing $\mathbf{G}$ for the Gram matrix $\mathbf{Q}$, only to materialize it later to compute the vector $\mathbf{G}\Delta\theta_b \in \mathbb{R}^B$.

Let the parameter update proposed by the underlying optimizer be $\Delta\theta_b$. To form the defining quantities of the optimization problem, we explicitly invoke the underlying optimizer (e.g., SGD with momentum or AdamW) to compute $\Delta\theta_b$, but only for the layers involved in the approximation—which we advocate to be solely the final layer ($\ell = L$). We emphasize that the gradients fed to the optimizer are the standard batch-averaged gradients, efficiently calculated via a regular backward pass. Partitioning this update into the weight component $\Delta\theta_{b,w} \in \mathbb{R}^{d_{out}d_{in}}$ (in flattened format) and the bias component $\Delta\theta_{b,b} \in \mathbb{R}^{d_{out}}$, we can then compute the product $\mathbf{G}\Delta\theta_b = \mathbf{G}_w\Delta\theta_{b,w} + \mathbf{G}_b\Delta\theta_{b,b}$ implicitly as:

$$\mathbf{G}\Delta\theta_b = ((\mathbf{x}\Delta\theta_{b,w}^\top) \odot \mathbf{g}_y) \mathbf{1} + \mathbf{g}_y\Delta\theta_{b,b}$$

where $\mathbf{1} \in \mathbb{R}^{d_{out}}$ is a vector of ones. In PyTorch, this translates to the following highly efficient operations:

Pseudocode 3. Implicit computation of $\mathbf{G}\Delta\theta_b$ without explicitly forming $\mathbf{G}$

```
# Dimensions: feat_all (x) [B, d_in], grad_logits_all (g_y) [B, d_out]
# dg_w [d_out, d_in], dg_b [d_out]
Gdg = (feat_all @ dg_w.t() * grad_logits_all).sum(dim=1)
Gdg += grad_logits_all @ dg_b
```

It is worth noting that if $\mathbf{G}$ were already computed and stored in memory, performing the direct matrix-vector multiplication (e.g., `grad_w_flat @ dg_w_flat`) is fast. In fact, for small layer dimensions (e.g., $d_{in} = 64, d_{out} \in \{10, 100\}$), the explicit method using the materialized $\mathbf{G}$ is usually a bit faster. However, when evaluating larger sizes like $d_{in} = 2048$ with $d_{out} \in \{100, 1000\}$, the implicit method is an order of magnitude faster and less susceptible to Out-of-Memory (OOM) errors. We note that our setup uses an NVIDIA Tesla T4 GPU with 16 GB, where OOM is triggered for the explicit baseline computing $\mathbf{G}$ when evaluating at $d_{in} = 2048, d_{out} = 1000$ for batch sizes $BS \geq 1024$. Thus, by avoiding the materialization of $\mathbf{G}$ everywhere, we retain excellent scaling capabilities across both memory footprint and runtime.

C. Guarding against instabilities

Before running the constrained optimization, we apply two lightweight safeguards to improve numerical robustness for the computed $\|\Delta\theta_b\|^2$, and $\mathbf{Q}$ for SGD and $\mathbf{Q}_1, \mathbf{Q}_2$ for AdamW. Since in practice we only use the last layer to approximate these quantities (i.e. $\|\Delta\theta_{b,L}\|^2, \mathbf{Q}_L, \mathbf{Q}_{L,1}, \mathbf{Q}_{L,2}$), it is possible that the computed values occasionally diverge from the actual global values, which can inadvertently affect optimization stability. Specifically, we found that $\|\Delta\theta_b\|^2$ may occasionally spike up, making the constraint of Equation (10) (for SGD) or Equation (14) (for AdamW) overly loose and distancing the update too much from the trusted region. Additionally, the proxy matrices (e.g., $\mathbf{Q}_L, \mathbf{Q}_{L,1}, \mathbf{Q}_{L,2}$) may also become ill-conditioned, necessitating a mechanism to regularize them so as to ensure our solver remains stable.

C.1. Safeguard 1: Spike control for $\|\Delta\theta_b\|^2$

Algorithm 2 Spike guard

- 1: At current iteration t compute $\|\Delta\theta_{b,L}\|^2, \mathbf{G}_L\Delta\theta_{b,L}$
 - 2: **if** $t = 1$: $\text{ema_dg} \leftarrow \|\Delta\theta_{b,L}\|^2 + 1$
 - 3: $\text{max_dg} \leftarrow \text{max_raise_dg} \cdot \text{ema_dg}$
 - 4: **if** $\|\Delta\theta_{b,L}\|^2 > \text{max_dg}$ **then**
 - 5: $\mathbf{G}_L\Delta\theta_{b,L} \leftarrow \frac{\sqrt{\text{max_dg}}}{\|\Delta\theta_{b,L}\|} \mathbf{G}_L\Delta\theta_{b,L}$ {Scaling down $\mathbf{G}_L\Delta\theta_{b,L}$ }
 - 6: $\|\Delta\theta_{b,L}\|^2 \leftarrow \text{max_dg}$ {Clipping before used in Equation (10) or (14)}
 - 7: **end if**
 - 8: $\text{ema_dg} \leftarrow \text{mom_dg} \cdot \text{ema_dg} + (1 - \text{mom_dg}) \cdot \|\Delta\theta_{b,L}\|^2$
-

The idea is to keep an exponential moving average (EMA), which we note as ema_dg , on the computed $\|\Delta\theta_{b,L}\|^2$. If at some iteration t we observe a more than max_raise_dg times increase of $\|\Delta\theta_{b,L}\|^2$ with respect to EMA value, then we will clip. In Algorithm 2 we assume that $t = 1$ is the first iteration. The reason behind adding the offset +1 in the initialization (see line 2) is because some training routines include a warm-up phase for the first iterations. During this phase the tiny learning rates make the $\|\Delta\theta_{b,L}\|$ also tiny. Without the offset +1 the ema_dg takes too many rounds to increase and this leads to unnecessarily many clippings. We used $\text{mom_dg} = 0.98$ and $\text{max_raise_dg} = 1.25$.

C.2. Safeguard 2: Avoiding ill-conditioning for $\mathbf{Q}$

Although the matrix $\mathbf{Q}$ is positive semi-definite, it can become highly ill-conditioned—either practically due to numerical precision limits, or natively due to rank deficiency in the extreme case where two per-sample gradients are linearly dependent. This resultant ill-conditioning leads to unstable updates. To prevent this, we monitor the condition number of $\mathbf{Q}$. Computing the exact condition number requires full spectral decomposition, which is computationally prohibitive as it has $\mathcal{O}(B^3)$ time complexity. Instead, we use the Lanczos algorithm to approximate the extremal eigenvalues of $\mathbf{Q}$ by projecting it onto a Krylov subspace. If I_L is the number of iterations of the algorithm, then a $I_L \times I_L$ symmetric tridiagonal matrix $\mathbf{T}$ is efficiently constructed, whose largest and smallest eigenvalues provide efficient approximations to those of $\mathbf{Q}$. The overall time complexity of this approximation is only $\mathcal{O}(I_L B^2)$ (dominated by I_L matrix-vector multiplications), representing a substantial speedup over exact decomposition since typically $I_L \ll B$. In practice, on an NVIDIA Tesla T4 GPU, we found that for smaller batch sizes ($B \leq 256$) computing the exact eigenvalues directly via standard PyTorch routines overhead remains negligible and is preferred. For larger batches where $\mathcal{O}(B^3)$ becomes noticeably slow, we switch to the Lanczos approximation, setting the number of iterations to $I_L = \min(\max(20, \lfloor B/32 \rfloor), 128)$.

Once the extremal eigenvalues ($\lambda_{\max}$ and $\lambda_{\min}$) are estimated, we evaluate the condition number $\kappa = \lambda_{\max}/\lambda_{\min}$. We maintain an exponential moving average (EMA) of the condition number, denoted $\bar{\kappa}$. If the current condition number exceeds a maximum allowed growth factor relative to the average ($\kappa > \text{max_raise_k} \cdot \bar{\kappa}$), we apply adaptive preconditioning by adding a scaled identity matrix $\alpha \mathbf{I}$ to $\mathbf{Q}$. The scalar α is analytically derived to precisely enforce the target condition number $\text{max_raise_k} \cdot \bar{\kappa}$. We used $\text{mom_k} = 0.98$ and $\text{max_raise_k} = 1.25$.

Algorithm 3 Extremal Eigenvalue Approximation (Lanczos)

```

1: Input: Symmetric matrix  $\mathbf{Q} \in \mathbb{R}^{B \times B}$ , iterations  $I_L$ .
2: Sample random vector  $\mathbf{v} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
3:  $\mathbf{v} \leftarrow \mathbf{v} / \|\mathbf{v}\|$ 
4:  $\mathbf{v}_{\text{prev}} \leftarrow \mathbf{0}, \beta_0 \leftarrow 0$ 
5: for  $i = 1$  to  $I_L$  do
6:    $\mathbf{w} \leftarrow \mathbf{Q}\mathbf{v}$ 
7:    $\alpha_i \leftarrow \mathbf{v}^\top \mathbf{w}$ 
8:    $\mathbf{w} \leftarrow \mathbf{w} - \alpha_i \mathbf{v} - \beta_{i-1} \mathbf{v}_{\text{prev}}$ 
9:    $\beta_i \leftarrow \|\mathbf{w}\|$ 
10:   $\mathbf{v}_{\text{prev}} \leftarrow \mathbf{v}$ 
11:   $\mathbf{v} \leftarrow \mathbf{w} / \beta_i$ 
12: end for
13: Construct a  $I_L \times I_L$  symmetric tridiagonal matrix  $\mathbf{T}$  with  $\alpha_1, \dots, \alpha_{I_L}$  on the main diagonal and  $\beta_1, \dots, \beta_{I_L-1}$  on the first sub-diagonal and first super-diagonal.
14: Compute extremal eigenvalues  $\lambda_{\min}, \lambda_{\max}$  of  $\mathbf{T}$ .
15: Output:  $\lambda_{\min}, \lambda_{\max}$ 

```

Algorithm 4 Adaptive Preconditioning of $\mathbf{Q}$

```

1: Input: Matrix  $\mathbf{Q}$ , EMA state  $\bar{\kappa}$ , momentum  $\text{mom\_k}$ , threshold multiplier  $\text{max\_raise\_k}$ .
2: if  $B \leq 256$  then  $\lambda_{\min}, \lambda_{\max}$  from exact_torch.linalg.eigvalsh(Q)
3: else  $\lambda_{\min}, \lambda_{\max} \leftarrow \text{Algorithm 3}(\mathbf{Q}, I_L = \min(\max(20, \lfloor B/32 \rfloor), 128))$ 
4:  $\kappa \leftarrow \lambda_{\max} / \lambda_{\min}$ 
5:  $\kappa_{\max} \leftarrow \text{max\_raise\_k} \cdot \bar{\kappa}$ 
6: if  $\kappa > \kappa_{\max}$  then
7:    $\alpha \leftarrow (\lambda_{\max} - \kappa_{\max} \lambda_{\min}) / (\kappa_{\max} - 1.0)$ 
8:    $\mathbf{Q} \leftarrow \mathbf{Q} + \alpha \mathbf{I}$ 
9:    $\kappa \leftarrow \kappa_{\max}$ 
10: end if
11:  $\bar{\kappa} \leftarrow \text{mom\_k} \cdot \bar{\kappa} + (1 - \text{mom\_k}) \cdot \kappa$ 
12: Output:  $\mathbf{Q}, \bar{\kappa}$ 

```

D. Solver and implementation details

D.1. Overview of the Chambolle–Pock algorithm

The Chambolle–Pock (CP) algorithm (Chambolle & Pock, 2011) is a first-order primal–dual method for convex optimization problems of the form:

$$\min_{\mathbf{w} \in \mathbb{R}^B} F(\mathbf{K}\mathbf{w}) + G(\mathbf{w}),$$

where $\mathbf{K} : \mathbb{R}^B \rightarrow \mathbb{R}^B$ is a linear operator and F, G are proper, convex (possibly non-smooth) functions. CP introduces a dual variable $\mathbf{y}$ associated with the linear term $\mathbf{K}\mathbf{w}$ and performs alternating proximal updates. The algorithm is particularly attractive when the proximal operators of G and F^* (the convex conjugate of F) are computationally inexpensive. Convergence is guaranteed provided the step sizes $\tau, \sigma > 0$ satisfy the condition $\tau\sigma\|\mathbf{K}\|_\sigma^2 < 1$, where $\|\mathbf{K}\|_\sigma$ denotes the spectral norm (largest singular value) of $\mathbf{K}$. The general update steps at iteration k are:

$$\begin{aligned} \mathbf{y}^{k+1} &= \text{prox}_{\sigma F^*}(\mathbf{y}^k + \sigma \mathbf{K} \tilde{\mathbf{w}}^k), \\ \mathbf{w}^{k+1} &= \text{prox}_{\tau G}(\mathbf{w}^k - \tau \mathbf{K}^\top \mathbf{y}^{k+1}), \\ \tilde{\mathbf{w}}^{k+1} &= \mathbf{w}^{k+1} + \theta(\mathbf{w}^{k+1} - \mathbf{w}^k). \end{aligned}$$

We now specialize this framework to our specific problem.

D.2. Problem statement

Our optimization problem is:

$$\mathbf{w}^* = \arg \min_{\mathbf{w} \in \mathcal{C}} H(-\mathbf{Q}_1 \mathbf{w} + \mathbf{q}) \quad (17)$$

where $\mathcal{C} = \{\mathbf{w} \in \mathbb{R}^B \mid \mathbf{w}^\top \mathbf{Q}_2 \mathbf{w} \leq r, \mathbf{0} \leq \mathbf{w} \leq \mathbf{t}\}$, $H(\mathbf{v}) = \sum_{i=1}^B \max(0, [\mathbf{v}]_i)$, $\mathbf{Q}_1, \mathbf{Q}_2 \in \mathbb{R}^{B \times B}$ are symmetric positive semi-definite matrices (for SGD case it is $\mathbf{Q}_1 = \mathbf{Q}_2$), $\mathbf{q} \in \mathbb{R}^B$, $r > 0$, $\mathbf{t} = t\mathbf{1}$ for some $t > 0$ and $\mathbf{1}$ a vector with all its elements equal to one. Let $I_{\mathcal{S}}(\mathbf{x})$ denote the indicator function of a set $\mathcal{S}$ in the context of convex analysis, i.e., $I_{\mathcal{S}}(\mathbf{x}) = 0$ if $\mathbf{x} \in \mathcal{S}$ and $+\infty$ otherwise. We map our problem to the CP template by choosing:

$$\mathbf{K} = -\mathbf{Q}_1, \quad F(\mathbf{z}) = H(\mathbf{z} + \mathbf{q}), \quad G(\mathbf{w}) = I_{\mathcal{C}}(\mathbf{w}) = I_{\{\mathbf{0} \leq \mathbf{w} \leq \mathbf{t}\}}(\mathbf{w}) + I_{\{\mathbf{w}^\top \mathbf{Q}_2 \mathbf{w} \leq r\}}(\mathbf{w})$$

D.3. Derivation of the updates

CP requires the proximal operator of the conjugate F^* and the proximal operator of G .

Conjugate of the (shifted) hinge sum. Given that the convex conjugate of $f_i(u) = \max(0, u + [\mathbf{q}]_i)$ with $u \in \mathbb{R}$ is:

$$f_i^*(y) = \sup_{u \in \mathbb{R}} \{uy - f_i(u)\} = \begin{cases} -[\mathbf{q}]_i y, & \text{if } y \in [0, 1], \\ +\infty, & \text{otherwise.} \end{cases}$$

the sum of conjugates:

$$F^*(\mathbf{y}) = \sum_{i=1}^B f_i^*([\mathbf{y}]_i) = I_{[0,1]^B}(\mathbf{y}) - \sum_{i=1}^B [\mathbf{y}]_i [\mathbf{q}]_i = I_{[0,1]^B}(\mathbf{y}) - \langle \mathbf{y}, \mathbf{q} \rangle,$$

where $\langle \mathbf{y}, \mathbf{q} \rangle$ is the inner product of the vectors $\mathbf{y}$ and $\mathbf{q}$.

Proximal of σF^* . The proximal operator of σF^* at $\mathbf{u}$ is defined as:

$$\begin{aligned} \text{prox}_{\sigma F^*}(\mathbf{u}) &= \arg \min_{\mathbf{y}} \left\{ \sigma (I_{[0,1]^B}(\mathbf{y}) - \langle \mathbf{y}, \mathbf{q} \rangle) + \frac{1}{2} \|\mathbf{y} - \mathbf{u}\|^2 \right\} \\ &= \arg \min_{\mathbf{y}} \left\{ 2\sigma I_{[0,1]^B}(\mathbf{y}) - 2\langle \mathbf{y}, \sigma \mathbf{q} \rangle + \|\mathbf{y}\|^2 - 2\langle \mathbf{y}, \mathbf{u} \rangle + \|\mathbf{u}\|^2 \right\} \\ &= \arg \min_{\mathbf{y}} \left\{ 2\sigma I_{[0,1]^B}(\mathbf{y}) + \|\mathbf{y}\|^2 - 2\langle \mathbf{y}, \mathbf{u} + \sigma \mathbf{q} \rangle \right\} \\ &= \arg \min_{\mathbf{y} \in [0,1]^B} \left\{ \|\mathbf{y} - (\mathbf{u} + \sigma \mathbf{q})\|^2 - \|\mathbf{u} + \sigma \mathbf{q}\|^2 \right\} \\ &= \arg \min_{\mathbf{y} \in [0,1]^B} \left\{ \|\mathbf{y} - (\mathbf{u} + \sigma \mathbf{q})\|^2 \right\} = \Pi_{[0,1]^B}(\mathbf{u} + \sigma \mathbf{q}). \end{aligned}$$

Note that the term $\|\mathbf{u} + \sigma \mathbf{q}\|^2$ is constant with respect to the minimization.

Proximal of G . The proximal operator is $\text{prox}_{\tau G}(\mathbf{v}) = \arg \min_{\mathbf{w}} (\tau G(\mathbf{w}) + \frac{1}{2} \|\mathbf{w} - \mathbf{v}\|^2)$. This actually amounts to an Euclidean projection on a convex set problem as:

$$\text{prox}_{\tau G}(\mathbf{v}) = \arg \min_{\mathbf{w} \in \mathcal{C}} \|\mathbf{w} - \mathbf{v}\|^2 = \Pi_{\mathcal{C}}(\mathbf{v}),$$

where the (convex) set $\mathcal{C}$ is the intersection of a box and an ellipsoid,

$$\mathcal{C} = \mathcal{B} \cap \mathcal{E}, \quad \mathcal{B} := [0, t]^B, \quad \mathcal{E} := \{\mathbf{w} \in \mathbb{R}^B : \mathbf{w}^\top \mathbf{Q}_2 \mathbf{w} \leq r\}.$$

Evaluating such proximal operators is often non-trivial (Combettes & Pesquet, 2011). A standard approach for projecting onto an intersection of convex sets is *Dykstra's projection algorithm* (Boyle & Dykstra, 1986), which alternates projections onto individual sets while incorporating correction vectors to ensure convergence to the true Euclidean projection $\Pi_{\mathcal{B} \cap \mathcal{E}}(\mathbf{w}_0)$. However, the exact projection onto the ellipsoid $\Pi_{\mathcal{E}}$ is computationally demanding, typically requiring an eigendecomposition of $\mathbf{Q}_2$ or an iterative root-finding procedure involving $B \times B$ linear systems, leading to an $\mathcal{O}(B^3)$ routine.

Since high precision is not critical within our CP iterations, we instead employ a computationally efficient alternating scheme (Bauschke & Borwein, 1996).

1. **Box Projection:** We first project onto $\mathcal{B}$ via element-wise clipping into $[0, t]^B$ (i.e. $\Pi_{\mathcal{B}}$).
2. **Radial Scaling:** If the resulting vector violates the ellipsoidal constraint (i.e., $\mathbf{w}^\top \mathbf{Q}_2 \mathbf{w} > r$), we scale it by a factor $\eta = \sqrt{r / (\mathbf{w}^\top \mathbf{Q}_2 \mathbf{w})}$.

This radial scaling provides a cheap $\mathcal{O}(B^2)$ alternative to the exact Euclidean projection. Crucially, by performing the radial scaling second, we ensure that the final point is guaranteed to lie in the feasible set $\mathcal{C}$. The opposite ordering cannot guarantee it.

D.4. Stopping criterion

Theoretical criterion: Primal-dual gap. A theoretically rigorous stopping criterion for primal-dual algorithms is the duality gap, defined as $\text{Gap}(\mathbf{w}, \mathbf{y}) = \text{Primal}(\mathbf{w}) - \text{Dual}(\mathbf{y})$, where Primal and Dual are the primal and dual objectives. The standard saddle-point form of our problem is:

$$\min_{\mathbf{w}} \max_{\mathbf{y}} \langle -\mathbf{Q}_1 \mathbf{w}, \mathbf{y} \rangle + G(\mathbf{w}) - F^*(\mathbf{y}).$$

The dual objective is given by $\text{Dual}(\mathbf{y}) = -F^*(\mathbf{y}) - G^*(\mathbf{Q}_1^\top \mathbf{y})$. While the conjugate $F^*(\mathbf{y}) = I_{[0,1]^B}(\mathbf{y}) - \mathbf{y}^\top \mathbf{q}$ is trivial to compute, the conjugate of G is not. The conjugate of $G(\mathbf{w}) = I_{\mathcal{C}}(\mathbf{w})$ is:

$$G^*(\mathbf{v}) = \sup_{\mathbf{w} \in \mathcal{C}} (\mathbf{v}^\top \mathbf{w}).$$

Evaluating $G^*(\mathbf{v})$ requires solving a maximization problem over the intersection of an ellipsoid and the non-negative orthant. This sub-problem is as computationally difficult as the original primal problem itself. Consequently, computing the exact duality gap at every iteration is computationally prohibitive.

Practical heuristic: Mean absolute update. Instead, we monitor convergence via the iterates in the original space. Let $\mathbf{w}^k$ denote the iterate in the primal domain at step k . We terminate the algorithm once the mean absolute update falls below a predefined tolerance:

$$\frac{1}{B} \sum_{i=1}^B |[\mathbf{w}^{k+1}]_i - [\mathbf{w}^k]_i| \leq \text{tol}. \quad (18)$$

D.5. Warm Start using Projected Gradient Descent

To accelerate convergence, we found that initializing the primal and dual variables using a few steps ($I_{PGD} = 10$) of projected (sub)gradient descent (PGD) is effective. In Algorithm 5 we use $\mathbb{1}(\cdot)$ to denote the standard binary indicator function that evaluates to 1 when its argument is true and 0 otherwise. With this notation, the (sub)gradient of the objective $H(\mathbf{x})$ can be compactly written as the element-wise boolean vector $\mathbb{1}(\mathbf{x} > \mathbf{0})$.

D.6. Efficient Spectral Norm Estimation

Computing the exact spectral norm $\|\mathbf{Q}_1\|_\sigma$ needed for defining the step sizes τ, σ using PyTorch’s standard routine (`torch.linalg.matrix_norm`) becomes a computational bottleneck as the batch size B grows, given its $\mathcal{O}(B^3)$ time complexity. Since the matrix $\mathbf{Q}_1$ is symmetric and positive semi-definite (PSD), its spectral norm coincides with its largest eigenvalue, $\lambda_{\max}$. As described in Section C, we already perform an efficient estimation of $\lambda_{\max}$ at each iteration using the Lanczos algorithm (or exact eigensolver for small B) to monitor and prevent ill-conditioning. Therefore, we reuse the $\lambda_{\max}$ output from the safeguarding check as the spectral norm $\|\mathbf{Q}_1\|_\sigma$.

D.7. Final algorithm

We use a safety factor of 0.5 (instead of 1.0) for the step sizes and 0.9 (again instead of 1.0) for the relaxation parameter to ensure numerical stability and dampen oscillations that might arise from the operator splitting approximation. In our implementation, the solver runs for a total of $I_{\text{solver}} = 50$ iterations, inclusive of the PGD warm-up steps.

Algorithm 5 Approximate projection onto Π_C

- 1: **Input:** $w, \mathbf{Q}_2, r$
 - 2: $w \leftarrow \Pi_{[0,2]^B}(w)$ {Box projection (component-wise clipping)}
 - 3: $w \leftarrow \sqrt{\frac{r}{\max(r, w^\top \mathbf{Q}_2 w)}} w$ {Radial scaling for ellipsoid constraint}
 - 4: **Output:** w
-

Algorithm 6 PGD Warm Start

- 1: **Input:** $\mathbf{Q}_1, \mathbf{Q}_2, \mathbf{q}, r, I_{PGD}, \eta$
 - 2: **if** $I_{PGD} = 0$ **then output:** $0, 0$
 - 3: $w = 0$
 - 4: $y = \mathbb{1}(\mathbf{q} > 0)$
 - 5: **for** $k = 1, \dots, I_{PGD}$ **do**
 - 6: $g \leftarrow -\mathbf{Q}_1^\top y$ {Subgradient of H }
 - 7: $w \leftarrow \text{Algorithm 5}(w - \eta g, \mathbf{Q}_2, r)$
 - 8: $y \leftarrow \mathbb{1}(-\mathbf{Q}_1 w + \mathbf{q} > 0)$ {Update active set}
 - 9: **end for**
 - 10: **Output:** w, y
-

Algorithm 7 Chambolle–Pock solver

- 1: **Input:** $\mathbf{Q}_1, \mathbf{Q}_2, \mathbf{q}, r, \text{tol}, I_{PGD}, I_{\text{solver}}, \lambda_{\max}$
 - 2: $\|\mathbf{Q}_1\|_\sigma \leftarrow \lambda_{\max}$ {Reused from Safeguard 2}
 - 3: $\tau \leftarrow 0.5 / \|\mathbf{Q}_1\|_\sigma$
 - 4: $\sigma \leftarrow 0.5 / \|\mathbf{Q}_1\|_\sigma$
 - 5: $w^1, y^1 \leftarrow \text{Algorithm 6}(\mathbf{Q}_1, \mathbf{Q}_2, \mathbf{q}, r, I_{PGD}, \tau)$ {Warm start}
 - 6: $\tilde{w}^1 = w^1$
 - 7: **for** $k = 1, \dots, I_{\text{solver}} - I_{PGD}$ **do**
 - 8: $y^{k+1} \leftarrow \Pi_{[0,1]^B}(y^k + \sigma(-\mathbf{Q}_1 \tilde{w}^k + \mathbf{q}))$ {Proximal of σF^* (Dual step)}
 - 9: $w \leftarrow w^k + \tau \mathbf{Q}_1^\top y^{k+1}$
 - 10: $w^{k+1} \leftarrow \text{Algorithm 5}(w, \mathbf{Q}_2, r)$ {Proximal of τG (Primal step)}
 - 11: $\tilde{w}^{k+1} \leftarrow w^{k+1} + 0.9(w^{k+1} - w^k)$ {Extrapolation}
 - 12: **if** $\frac{1}{B} \sum_{i=1}^B |w^{k+1} - w^k|_i \leq \text{tol}$ **then break**
 - 13: **end for**
 - 14: **Output:** w
-

E. Architectural and Training Details

The **ViT** architecture consists of 7 transformer layers with 12 attention heads. For the standard version, both the hidden and MLP dimensions are set to 384, while for the **ViT-small** variant we use a dimension of 192 for both; the only difference from the reference implementation of (omihub777, 2021) is that we utilize the AdamW optimizer instead of Adam.

For convolutional models on CIFAR datasets, we follow the **ResNet-44** (and the smaller **ResNet-20**) specifications for CIFAR (He et al., 2016), which utilize three stages of residual blocks with 16, 32, and 64 channels respectively. However, because PyTorch’s `vmap` does not support Batch Normalization, we replace it with Group Normalization (GN) (Wu & He, 2018) using 8 groups across all stages to enable efficient per-sample gradient computation.

For both ResNet and ViT architectures, we adopt a baseline batch size of 128. The initial learning rate is 0.1 for the former (using SGD optimizer) and 0.001 for the latter (using AdamW optimizer). When the batch size is increased, we keep the total number of epochs constant, and we adjust the learning rate according to the square root scaling rule (Hoffer et al., 2017):

$$\text{lr}_{\text{new}} = \text{lr}_{\text{base}} \sqrt{\frac{B_{\text{new}}}{B_{\text{base}}}}, \quad (19)$$

where $B_{\text{base}} = 128$ and lr_{base} is the corresponding baseline learning rate. The rest of the hyperparameters match those in (omihub777, 2021) for ViT and/or (He et al., 2016) for ResNet.

For Tiny-ImageNet, a downsampled ImageNet-1K variant with 200 classes and RGB images of size 64×64 , we train ResNet-50 using the same recipe as in (He et al., 2016). For ConvNeXt-T, we follow the training recipe of (Liu et al., 2022) and report the accuracy of the exponential moving average (EMA) model, using FP16 training to avoid memory issues. For ImageNet-100, we again start from the ConvNeXt recipe but reduce the number of epochs from 300 to 100 and, accordingly, the number of warmup epochs from 20 to 5. In this setting, we report the non-EMA model accuracy: the EMA coefficient used in (Liu et al., 2022) is tied to the original number of training iterations, and becomes too slow to adapt when training with fewer classes and fewer epochs.

For CIFAR-10, CIFAR-100, and Tiny-ImageNet, all reported results are averaged over 5 random seeds. For ImageNet-100, we report results from a single seed.

F. Additional figures on the validity of the approximation and update quality

In this section, we provide the derivation for the optimal scaling factor α^* and present supplementary figures further validating our approximation strategy. As detailed in the main text, we assess performance via two metrics: (1) the *structural approximation error*, measured by the relative Frobenius norm $\|\mathbf{Q} - \alpha^* \mathbf{Q}_{\ell:L}\|_F / \|\mathbf{Q}\|_F$ for experiments with SGD optimizer and $\|\mathbf{Q}_1 - \alpha^* \mathbf{Q}_{1,\ell:L}\|_F / \|\mathbf{Q}_1\|_F$ for experiments with AdamW; and (2) the *relative effective objective reduction* (REOR), defined as $(J(\Delta\theta_b) - J(\Delta\theta^*)) / J(\Delta\theta_b)$. This second metric quantifies the reduction of the original per-sample interference objective (Equation (2)), where $\Delta\theta^* = \Delta\theta_b + \mathbf{G}^\top \mathbf{w}^*$ and $\mathbf{w}^*$ is the solution returned by the solver.

F.1. Derivation of the Optimal Scaling Factor

We seek the scalar α^* that minimizes the Frobenius distance between the full Gram matrix $\mathbf{Q}$ and the scaled partial Gram matrix $\mathbf{Q}_{\ell:L}$. The optimization problem is:

$$\alpha^* = \arg \min_{\alpha \in \mathbb{R}} \|\mathbf{Q} - \alpha \mathbf{Q}_{\ell:L}\|_F^2$$

Expanding the squared norm:

$$\begin{aligned} \|\mathbf{Q} - \alpha \mathbf{Q}_{\ell:L}\|_F^2 &= \langle \mathbf{Q} - \alpha \mathbf{Q}_{\ell:L}, \mathbf{Q} - \alpha \mathbf{Q}_{\ell:L} \rangle_F \\ &= \|\mathbf{Q}\|_F^2 - 2\alpha \langle \mathbf{Q}, \mathbf{Q}_{\ell:L} \rangle_F + \alpha^2 \|\mathbf{Q}_{\ell:L}\|_F^2 \end{aligned}$$

where $\langle \mathbf{A}, \mathbf{B} \rangle_F = \text{Tr}(\mathbf{A}^\top \mathbf{B})$ is the Frobenius inner product. Taking the derivative with respect to α and setting it to zero yields:

$$\frac{d}{d\alpha} = -2\langle \mathbf{Q}, \mathbf{Q}_{\ell:L} \rangle_F + 2\alpha \|\mathbf{Q}_{\ell:L}\|_F^2 = 0$$

Solving for α :

$$\alpha^* = \frac{\langle \mathbf{Q}, \mathbf{Q}_{\ell:L} \rangle_F}{\|\mathbf{Q}_{\ell:L}\|_F^2} = \frac{\text{Tr}(\mathbf{Q}^\top \mathbf{Q}_{\ell:L})}{\|\mathbf{Q}_{\ell:L}\|_F^2}$$

Since $\mathbf{Q}$ and $\mathbf{Q}_{\ell:L}$ are positive semi-definite (and non-zero), both the numerator and denominator are positive, ensuring $\alpha^* > 0$. Substituting $\mathbf{Q}$ with $\mathbf{Q}_1$ and $\mathbf{Q}_{\ell:L}$ with $\mathbf{Q}_{1,\ell:L}$ gives the corresponding result for the AdamW case.

F.2. Additional Results

Consistent with the main text, the top rows of the following figures display the structural error (lower is better), while the bottom rows display the distribution of the effective objective reduction (higher/positive is better).

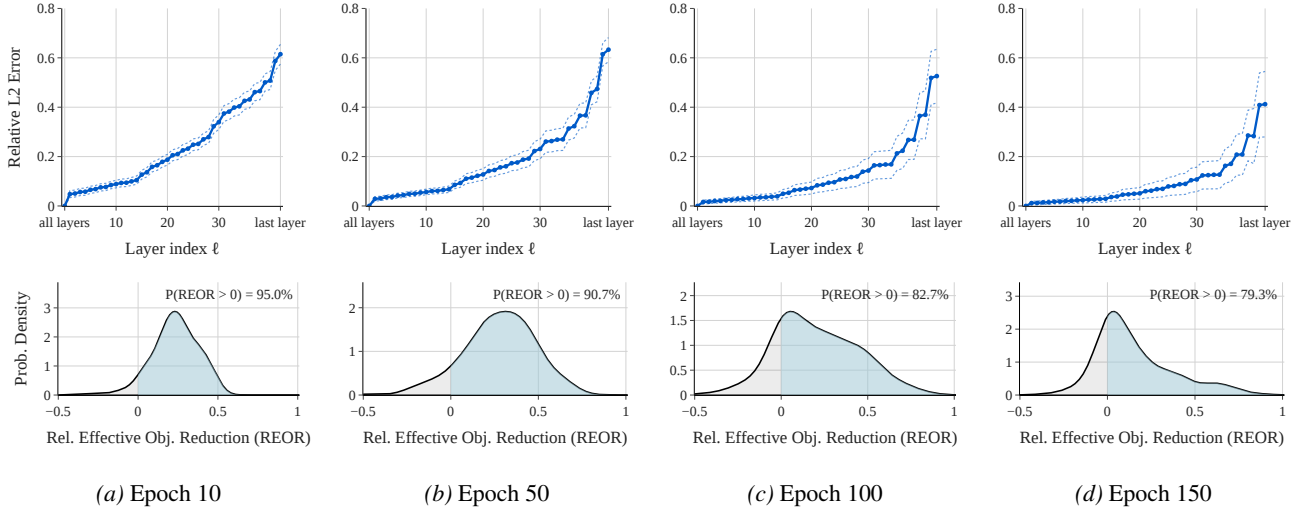


Figure 3. Approximation quality for ResNet-20 on CIFAR-10 for Batch Size = 128 at different training epochs. **Top Row:** Relative L_2 error of the Gram matrix approximation $\mathbf{Q}$. **Bottom Row:** Distribution of the relative effective objective reduction (REOR).

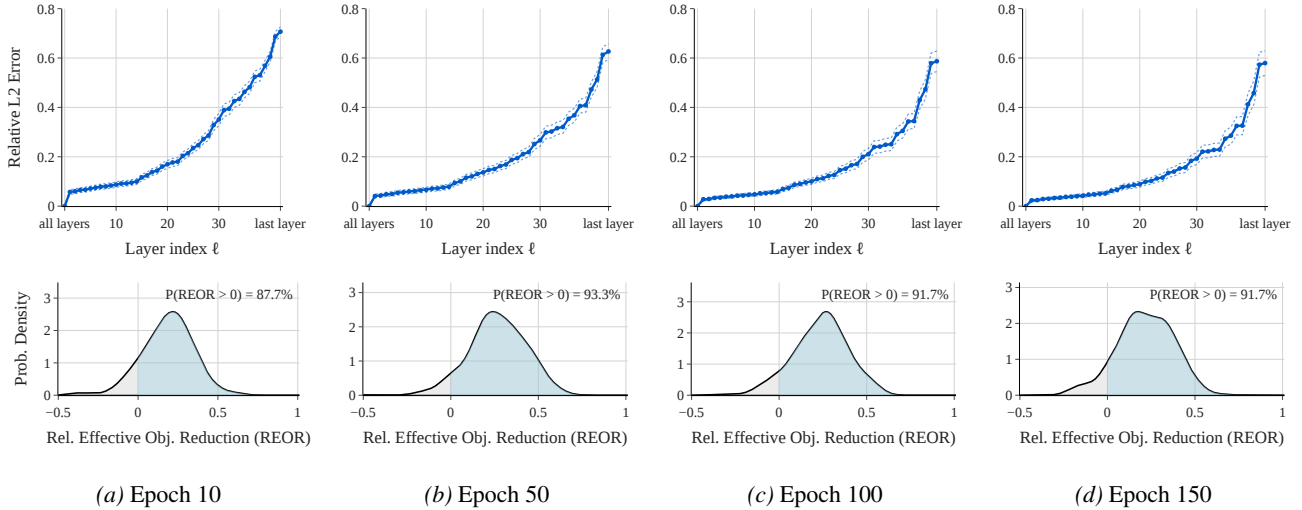


Figure 4. Approximation quality for ResNet-20 on CIFAR-100 for Batch Size = 128 at different training epochs. **Top Row:** Relative L_2 error of the Gram matrix approximation $\mathbf{Q}$. **Bottom Row:** Distribution of the relative effective objective reduction (REOR).

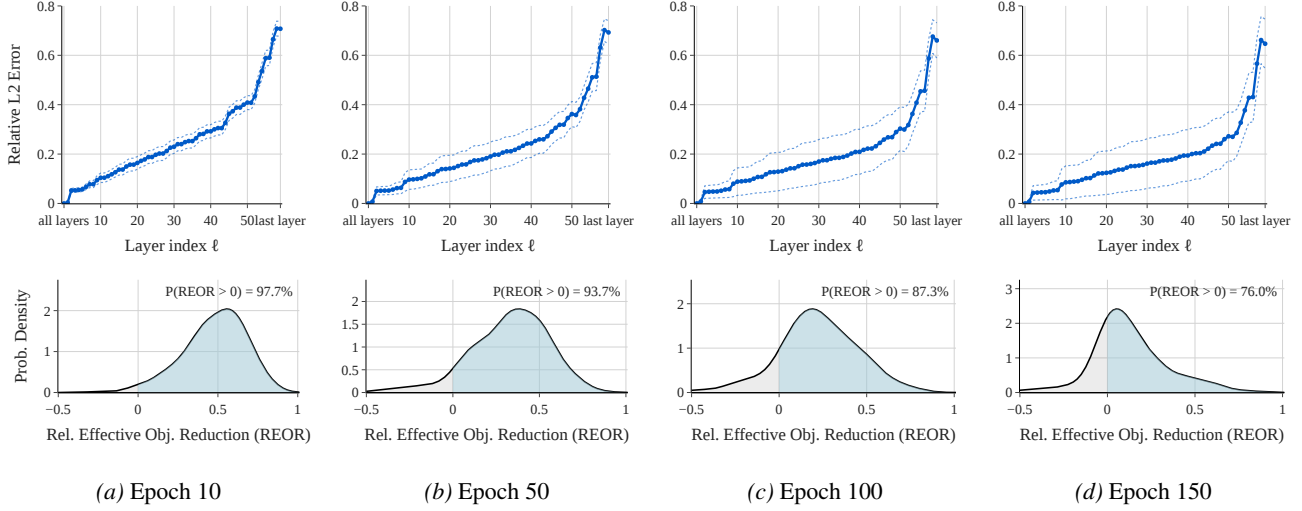


Figure 5. Approximation quality for ViT-small on CIFAR-10 for Batch Size = 128 at different training epochs. **Top Row:** Relative L_2 error of the Gram matrix approximation $\mathbf{Q}_1$. **Bottom Row:** Distribution of the relative effective objective reduction (REOR).

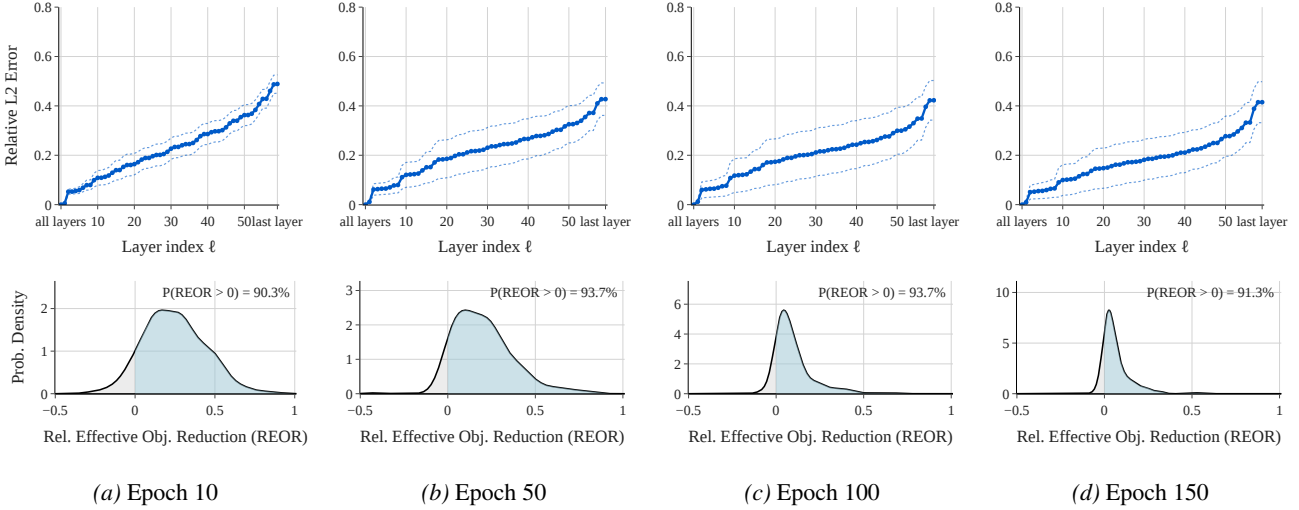


Figure 6. Approximation quality for ViT-small on CIFAR-100 for Batch Size = 128 at different training epochs. **Top Row:** Relative L_2 error of the Gram matrix approximation $\mathbf{Q}_1$. **Bottom Row:** Distribution of the relative effective objective reduction (REOR).

G. Additional figures on the efficiency of the solver

We present additional results demonstrating the efficiency of the proposed solver (detailed in Section D) on the optimization problem (17). We evaluate the solver’s performance based on convergence speed and the quality of the resulting parameter solutions. Mirroring the main analysis, solution quality is quantified via the *relative solver objective reduction*, defined as:

$$\frac{H(\mathbf{q}) - H(-\mathbf{Q}_X \mathbf{w}^* + \mathbf{q})}{H(\mathbf{q})} \quad (20)$$

where $\mathbf{Q}_X$ corresponds to the matrix $\mathbf{Q}_L$ for SGD (Equation (10)) or $\mathbf{Q}_{1,L}$ for AdamW (Equation (14)) and $\mathbf{w}^*$ is the solver’s output optimal solution.

The following figures illustrate these convergence metrics across varying batch sizes. In each pair, the top plot highlights the relative objective reduction (interference avoided), while the bottom plot tracks the number of iterations required by the CP solver to reach a solution. The maximum allowed number of iterations our solver may use is set at $I_{\text{solver}} = 50$.

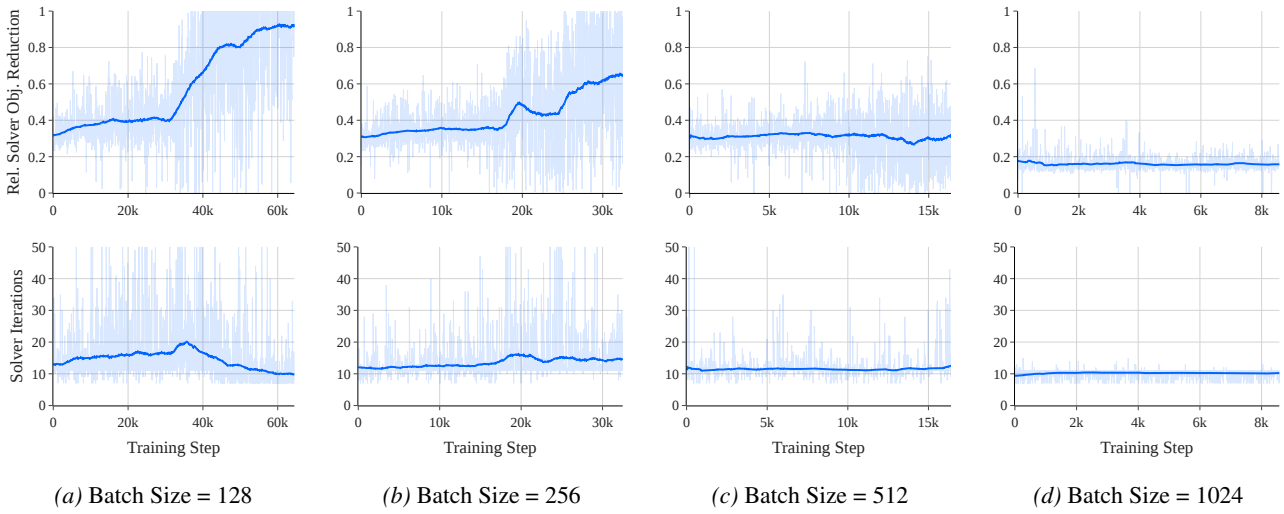


Figure 7. Relative objective reduction (top) and number of solver iterations (bottom) across different batch sizes for ResNet-44 on CIFAR-10.

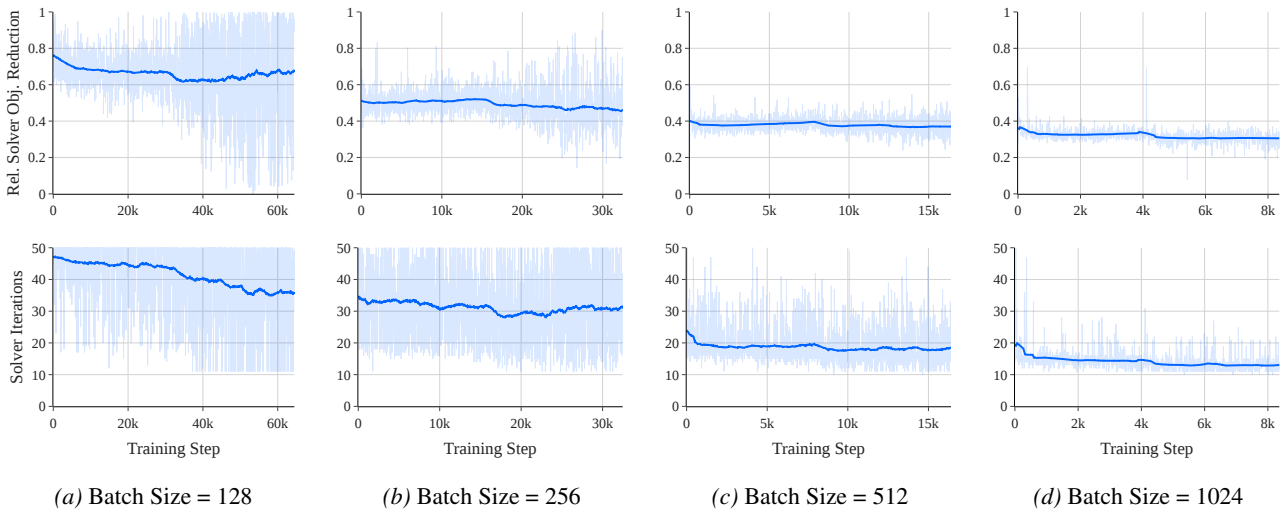


Figure 8. Relative objective reduction (top) and number of solver iterations (bottom) across different batch sizes for ResNet-44 on CIFAR-100.

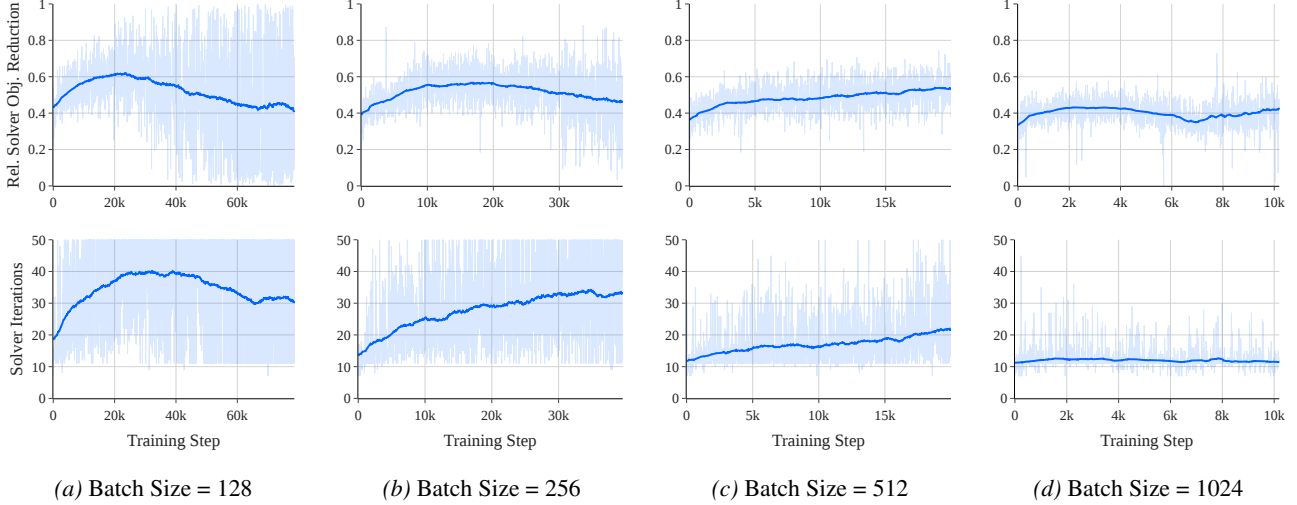


Figure 9. Relative objective reduction (top) and number of solver iterations (bottom) across different batch sizes for ViT-c on CIFAR-10.

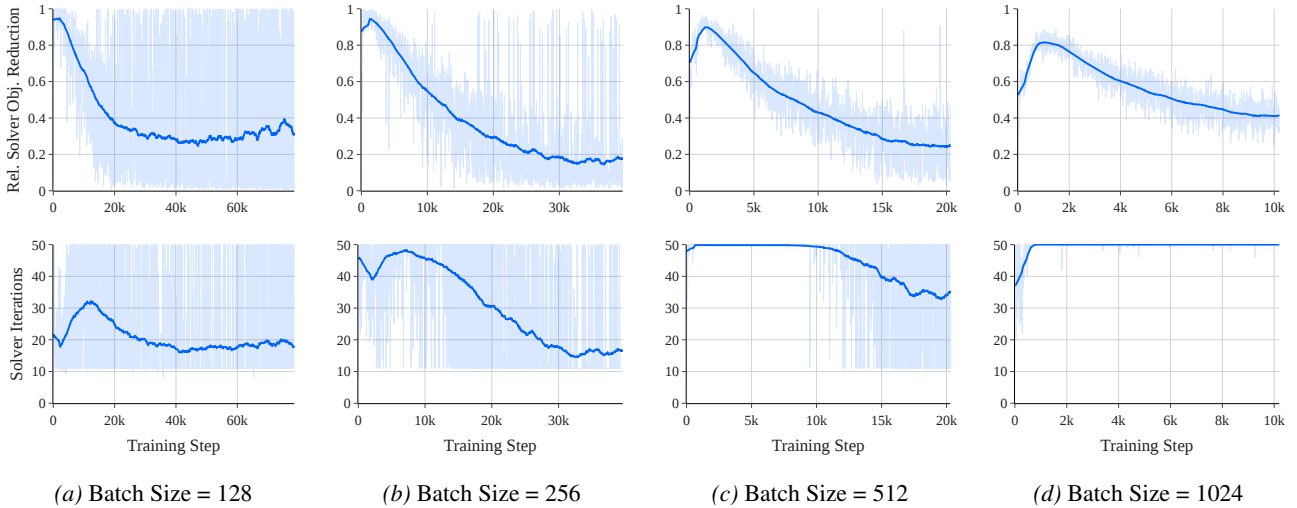


Figure 10. Relative objective reduction (top) and number of solver iterations (bottom) across different batch sizes for ViT-c on CIFAR-100.

H. Generalized Multi-Layer Algorithm and Precision Analysis

Algorithm 8 details the generalized training iteration using SGD when expanding the approximation to include multiple layers ($\ell < L$). We decompose the architecture as $F(\mathbf{x}; \boldsymbol{\theta}) = F_{\ell:L}(F_{1:\ell-1}(\mathbf{x}; \boldsymbol{\theta}_{1:\ell-1}); \boldsymbol{\theta}_{\ell:L})$. Because intermediate blocks contain also non-linear operations, we explicitly compute the per-sample gradients $\mathbf{G}_{\ell:L}$ using PyTorch’s `vmap`. For notational clarity, we treat the quantities $\mathbf{G}_{\ell:L}$ and $\nabla_{y_\ell} \mathcal{L}$ as flattened two-dimensional tensors of shape $[B, D]$, though in practice they may possess arbitrary trailing dimensions. The multiplication with the weight vector $(\mathbf{1} + \mathbf{w}^*)$ denotes a weighted sum across the batch dimension. In modern deep learning frameworks, this can be efficiently implemented using Einstein summation (e.g., `einsum('b...,b->...', tensor, weights)`). The extension to AdamW is straightforward with the difference being that the optimal $\mathbf{w}^*$ is obtained by solving Equation (14) instead of Equation (10).

Impact of Per-Sample Gradients on Precision: We investigate the numerical stability of explicitly computing per-sample gradients via `vmap`. In Figure 11, we show the effect of increasing the number of blocks handled by `vmap` (i.e., decreasing ℓ). For this experiment, we set $\mathbf{w} = \mathbf{0}$, making the parameter update theoretically equivalent to standard SGD. As expected, time per batch and peak memory increase substantially as more blocks are included. However, we also observe fluctuations in test accuracy depending on the approximation depth. Theoretically, these fluctuations should not occur: all experiments use

Algorithm 8 Generalized Training Iteration for SGD (Multi-Layer)

- 1: **Input:** Batch $\mathbf{x}$
- 2: $y_\ell \leftarrow F_{1:\ell-1}(\mathbf{x}; \boldsymbol{\theta}_{1:\ell-1})$
- 3: $\mathcal{L} \leftarrow \mathcal{L}(F_{\ell:L}(y_\ell; \boldsymbol{\theta}_{\ell:L}))$
- 4: Compute per-sample gradients $\mathbf{G}_{\ell:L}$ and $\nabla_{y_\ell} \mathcal{L}$
- 5: $\Delta \boldsymbol{\theta}_{b,\ell:L} \leftarrow \text{SGD}(\mathbf{G}_{\ell:L}^\top \mathbf{1})$
- 6: Solve Equation (10) for $\mathbf{w}^*$ {Algorithm 7 in Section D}
- 7: $\tilde{\mathbf{g}}_{\ell:L} \leftarrow \mathbf{G}_{\ell:L}^\top (\mathbf{1} + \mathbf{w}^*)$
- 8: $\tilde{\delta}_{y_\ell} \leftarrow (\nabla_{y_\ell} \mathcal{L})^\top (\mathbf{1} + \mathbf{w}^*)$
- 9: $\tilde{\mathbf{g}}_{1:\ell-1} \leftarrow \text{backward}(\tilde{\delta}_{y_\ell}, F_{1:\ell-1})$
- 10: Execute an $\text{SGD}([\tilde{\mathbf{g}}_{1:\ell-1}, \tilde{\mathbf{g}}_{\ell:L}])$ step

identical random seeds, and summing per-sample gradients is mathematically equivalent to computing the batch-averaged gradients directly. Their presence indicates that explicitly materializing and summing per-sample gradients over numerous layers can introduce numerical imprecisions compared to the standard, fused backward pass. This finding provides further motivation for our design choice: restricting the per-sample gradient computation exclusively to the final linear layer as it minimizes not only computational and memory overheads but also potential numerical drift.

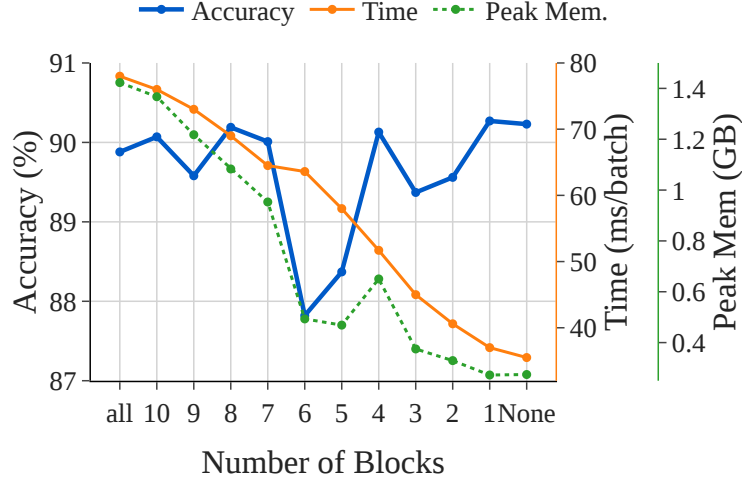


Figure 11. Efficiency comparison for ResNet-20-gn (accuracy, time per batch, and peak memory across number of blocks).